

B. Tech Mini-Project Final Report

MAZE SOLVING MICRO-MOUSE

**Submitted in partial fulfillment for the Degree of
Bachelor of Technology in
Artificial Intelligence and Machine Learning**

Submitted by

**Aison Yesudas (MCE23AIM008)
Alan Francis (MCE23AIM011)
Nikhil Vinod (MCE23AIM046)
Rajan Athul Raj (MCE23AIM050)**

Under the guidance of
Ms. Aswathy T K



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**

**MARIAN ENGINEERING COLLEGE
TRIVANDRUM**

April 2026

MAZE SOLVING MICRO-MOUSE

The Mini Project report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Artificial Intelligence & Machine Learning

by

Aison Yesudas (Reg. No. MCE23AIM008)

Alan Franics (Reg. No. MCE23AIM011)

Nikhil Vinod (Reg. No. MCE23AIM046)

Rajan Athul Raj (Reg. No. MCE23AIM050)



**Marian Engineering College
Trivandrum
Kerala**

April 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Marian Engineering College
Kazhakuttom, Trivandrum - 695582



CERTIFICATE

This is to certify that this report entitled “MAZE SOLVING MICRO-MOUSE” by NIKHIL VINOD (MCE23AIM0046), RAJAN ATHUL RAJ (MCE23AIM050), AISON YESUDAS (MCE23AIM008), and ALAN FRANCIS (MCE23AIM011) to the APJ Abdul Kalam Technological University in partial fulfillment of the B. Tech Degree in Artificial Intelligence and Machine Learning is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other university or institute for any purpose.

PROJECT GUIDE

Ms. Aswathy T K
Assistant Professor
Dept. of AI & ML

PROJECT COORDINATOR

Dr. Aswathy S U
Professor
Dept. of AI & ML

HEAD OF THE DEPARTMENT

Prof. Balu John
Professor and HOD
Dept. of AI & ML

ACKNOWLEDGEMENT

We are extremely thankful to our Principal, **Dr. Abdul Nizar M**, for granting us permission and providing the necessary facilities to carry out this project. We express our heartfelt thanks to **Prof. Balu John**, Head of the Department of Artificial Intelligence & Machine Learning, for his valuable guidance and continuous support throughout this work. We are deeply grateful to our Project Coordinators, **Dr. Aswathy S. U**, **Ms. Aswathy T. K**, and **Ms. Jyotsna A**, for their guidance, support, and encouragement at every stage of this project. We extend our sincere thanks to our project guide, **Ms. Aswathy T. K**, Assistant Professor, Department of Artificial Intelligence & Machine Learning, for her constant support, timely advice, and valuable guidance during the development and presentation of this project. We also express our sincere gratitude to God Almighty for His blessings and strength that enabled us to successfully complete this work. Finally, we thank all our friends and well-wishers for their support, encouragement, and cooperation throughout this endeavour.

ABSTRACT

This project focuses on the design and implementation of a **Micromouse**, an autonomous mobile robot capable of solving a maze without human intervention. The problem, though studied for decades, remains significant in robotics due to its requirement for efficient decision-making and path planning. The system begins with simple wall-following techniques and progressively incorporates more advanced algorithms to improve accuracy and efficiency. The primary emphasis is on the Flood-Fill algorithm, which enables the robot to generate an internal map of the maze and determine the shortest path to the goal. Using sensor inputs to detect walls and obstacles, the robot continuously updates its navigation strategy. This project provides a structured approach to understand robot intelligence and contributes to the development of efficient pathfinding mechanisms in autonomous robotic systems.

Contents

List of Figures	iii
List of Tables	iv
1 Introduction	1
1.1 Introduction	2
1.2 Motivation	2
1.3 Problem Definition	2
1.4 Objectives	3
1.5 Scope of the project	4
1.6 Chapter Details	4
2 Literature Survey	5
2.1 Maze Solving with Image-Based Mapping	6
2.2 Bruins Micromouse Flood Fill Module	7
2.3 The Maze Problem Solved by Micromouse	7
2.4 Maze Solving Robot Using Arduino Uno	8
2.5 Design of a Maze-Solving Mouse	9
3 Proposed System	10
3.1 System Overview	10
3.1.1 System Architecture	11
3.1.2 Working Principle	12
3.1.3 Flood-Fill Algorithm (Mathematical Representation)	13
3.1.4 Example Based on Figure	13
3.1.5 Working Steps	14
3.1.6 Algorithm Design	14
3.1.7 Algorithm Description	14
3.1.8 Pseudo Code	15
3.1.9 Working Explanation	15
3.1.10 Time Complexity	15
3.1.11 Space Complexity	16
3.2 Simulation of Maze	17
3.2.1 Maze Construction	17
3.2.2 Maze Representation	18
3.2.3 Flood-Fill Implementation	18
3.2.4 Path Extraction	18
3.2.5 Visualization	19

3.2.6	Mapping & Navigation	20
3.2.7	Maze Mapping	20
3.3	Distance Mapping using Flood-Fill	21
3.3.1	Navigation Strategy	21
3.3.2	Dynamic Adaptation	21
3.3.3	Path Optimization	21
3.3.4	Flowchart	22
3.3.5	Advantages	24
4	Tools & Technologies Used	25
4.1	Hardware Components	26
4.1.1	Arduino Nano	26
4.1.2	Infrared (IR) Sensors	27
4.1.3	Motor Driver (TB6612FNG)	28
4.1.4	DC Motors	29
4.1.5	Power Supply (2 AA Rechargeable Batteries)	31
4.1.6	Jumper Wires	32
4.2	Software Tools	32
4.2.1	Arduino IDE	32
4.2.2	Embedded Programming using C/C++ (.ino)	32
4.2.3	Python (Simulation)	33
4.2.4	Flood-Fill Algorithm	33
4.2.5	Grid-Based Mapping	33
4.2.6	Sensor-Based Navigation	34
4.3	Summary	34
5	Testing Strategies & Result Achieved	35
5.1	Testing Strategies	35
5.1.1	Simulation Testing	35
5.2	Results Achieved	38
5.2.1	Performance Analysis	40
5.3	Summary of Results	42
6	Conclusion & Future Work	43
6.1	Conclusion	43
6.2	Future Work	44
	Bibliography	45
	Appendix A Code Part	47
A.1	Arduino Implementation Code	47
A.2	Simulation Code for Maze Creation	55
A.3	Simulation Code for Flood-Fill Algorithm	57

List of Figures

1.1	Micromouse Maze Structure	1
3.1	System Architecture of Micromouse Robot (Arduino Nano Based) . .	11
3.2	Flooding the Maze with numbers.	12
3.3	Flood-Fill Distance Map and Path Selection	13
3.4	Time and Space Complexity	16
3.5	Simulation of the maze and the working of the algorithm	17
3.6	Mapping and Navigation	20
3.7	Workflow of Maze Solving Micro-Mouse	23
4.1	Microcontroller Board	26
4.2	Sensor Module	27
4.3	Driver Module	28
4.4	DC Motor Used in the Micromouse Robot	29
4.5	Robot Car Chassis Structure	30
4.6	Power Module	31
4.7	wires(F-F)	32
5.1	Micromouse Robot Prototype from Different Views	39

List of Tables

5.1	Performance Analysis of Micromouse System	41
-----	---	----

Chapter 1

Introduction

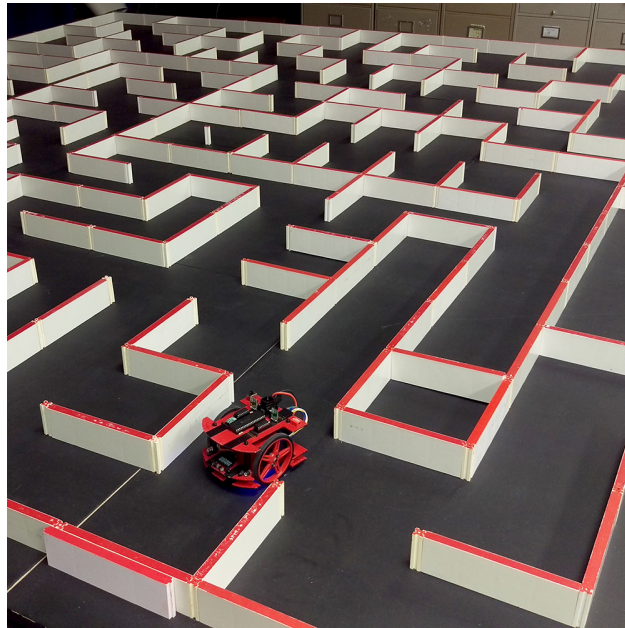


Figure 1.1: Micromouse Maze Structure

A Micromouse is a small autonomous mobile robot designed to navigate and solve a maze without human intervention. The standard Micromouse challenge consists of a 8×8 grid-based maze where the robot must travel from a predefined starting position to the central goal region. The primary objective is not only to reach the goal but to determine and traverse the shortest possible path in the least amount of time. This problem has been widely studied for decades and remains an important benchmark in the field of robotics and same as artificial intelligences, as it involves core concepts such as perception, decision-making, and path planning. To successfully solve the maze, the robot must be equipped with sensors to detect walls and obstacles, along with a control system capable of processing this information in real time. As the robot explores the maze, it incrementally builds an internal representation or map of the environment. Based on this map, it applies path-finding algorithms to determine the optimal route to the goal.

1.1 Introduction

A Micromouse designed to navigate and solve a maze. Among the various algorithms available, the Flood-Fill algorithm is one of the most widely used techniques in Micromouse competitions. It operates by assigning distance values to each cell in the maze, starting from the goal and propagating outward, thereby enabling the robot to determine the shortest path from any given position. This method guarantees an optimal path when the maze is fully known. While finding the shortest path is computationally straightforward using algorithms like Flood-Fill, the real challenge lies in efficiently exploring the maze and accurately mapping unknown environments.

Factors such as incomplete information, sensor inaccuracies, and movement add complexity to the problem. Additionally, optimizing the robot's motion for speed, smooth turns, and minimal energy consumption further enhances performance. This project focuses on the design and implementation of a Micromouse system using the Flood-Fill algorithm for efficient maze solving.

It aims to provide a sys approach for understanding autonomous navigation, decision-making algorithms, and real-time robotic control, thereby contributing to the development of intelligent robotic systems..

1.2 Motivation

The motivation behind the Micromouse project is to explore , develop an autonomous system capable of perceiving its environment, making intelligent decisions, navigating efficiently without human intervention. This project provides hands-on experience in integrating sensors, embedded systems, and decision-making algorithms. From an artificial intelligence perspective, the project emphasizes the development of an intelligent agent that can operate in an unknown environment, build an internal representation (map), and make optimal decisions based on available data.

The use of algorithms such as Flood-Fill highlights essential AI concepts including problem-solving, search strategies, and path optimization. Furthermore, the project is motivated by its relevance to real-world applications such as autonomous vehicles, robotic navigation systems, and smart assistants, where efficient decision-making and environment understanding are critical..

1.3 Problem Definition

The problem addressed in this project is the design and implementation of a robot capable of efficiently navigating and solving a maze.

The maze is typically structured as a grid of cells (commonly 8×8), with unknown wall configurations. The robot starts from a predefined initial position and must reach the goal region, usually located at the center of the maze. The primary challenge lies in enabling the robot to operate in an unknown environment where no prior information about the maze layout is available. The robot must rely on its onboard sensors to detect walls and obstacles in real time. Based on this sensory input, it must incrementally construct an internal representation (map) of the maze while simultaneously making navigation decisions. Another key aspect

of the problem is determining the shortest or most optimal path to the goal. The robot must implement an efficient path-planning algorithm, such as the Flood-Fill algorithm, to calculate the minimum distance from its current position to the target. This involves continuously updating distance values and selecting the best possible direction at each step. In addition to path planning, the robot must handle practical constraints such as sensor inaccuracies, limited computational resources, and physical movement limitations. It must ensure smooth navigation, avoid collision, and optimize its motion for speed and accuracy. Furthermore, the robot is expected not only to reach the goal but also to improve its performance by refining the discovered path and minimizing traversal time. This introduces additional complexity in balancing exploration and optimization. Thus, the problem involves integrating perception, mapping, decision-making, and control into a single system capable of autonomously solving a maze in an efficient and reliable manner.

1.4 Objectives

The primary objective of this project is to design and develop an autonomous robot capable of efficiently navigating and solving a maze using intelligent path-planning techniques. The project aims to provide a comprehensive understanding of systems, including maze structure, navigation strategies, control mechanisms, and real-time decision-making processes.

An important objective is to simulate the maze environment and robot behavior using Python, which allows for the testing, analysis, and validation of different algorithms in a controlled virtual setup before actual hardware implementation. This simulation helps in understanding movement logic, path optimization, and system performance under various maze configurations. The project is the design and implementation of the Flood-Fill algorithm, which enables the robot to compute the shortest path from any position to the goal by dynamically updating distance values. The project also aims to develop an efficient maze mapping system, where the robot uses IR sensors to detect walls and obstacles, constructs an internal representation of the maze, and continuously updates this map as it explores unknown regions. In addition to software development, the project involves the design and construction of a physical maze based on standard Micromouse dimensions, ensuring proper wall height, thickness, and cell structure for accurate sensing and navigation. Furthermore, the project includes the design and fabrication of the Micromouse robot using hardware components such as the Arduino Nano microcontroller, TB6612FNG motor driver, DC motors, and IR sensors, ensuring proper integration of all subsystems. Finally, the objective is to integrate both hardware and software components into a unified system capable of real-time operation, enabling the robot to make intelligent decisions, navigate smoothly, avoid collisions, and optimize its path for minimum time and maximum efficiency. The project also aims to evaluate system performance in terms of accuracy, speed, reliability, and adaptability, thereby providing a strong foundation for further advancements in autonomous robotics and artificial intelligence.

1.5 Scope of the project

The scope of this project focuses on the design, simulation, and implementation of an autonomous Micromouse robot capable of solving a maze using efficient path-planning techniques. The project includes the development of a virtual simulation environment using Python to test and analyze the behavior of the robot and validate algorithms such as Flood-Fill for shortest path determination.

It also covers the design and construction of a physical maze based on standard dimensions, ensuring compatibility with sensor-based navigation. The scope further extends to the hardware implementation of the Micromouse robot using components such as the Arduino Nano microcontroller, TB6612FNG motor driver, DC motors, and IR sensors for wall detection. The project involves integrating these components to enable real-time sensing, mapping, and navigation within the maze. However, the project is limited to basic autonomous navigation and shortest path finding using the Flood-Fill algorithm. Advanced features such as high-speed optimization, dynamic path smoothing, machine learning-based decision-making, and complex multi-agent coordination are beyond the scope of this work. The primary focus remains on achieving reliable maze solving with accurate mapping and efficient path planning, providing a strong foundation for future enhancements in intelligent robotic systems.

1.6 Chapter Details

The report is organised as follows:

- Chapter 1 provides an introduction to the Micromouse project, including the motivation, problem definition, objectives, scope of the project, and overall structure of the report.
- Chapter 2 presents the literature survey, which includes a review of various research works and existing approaches related to maze-solving techniques and path-planning algorithms.
- Chapter 3 describes the proposed system in detail. It includes the methodology adopted, the system architecture diagram and its description, module design, and data flow diagrams representing the working of the system.
- Chapter 4 discusses the tools and technologies used for the development of the project, including software and hardware components.
- Chapter 5 presents the testing strategies and results achieved. It includes the results obtained from both simulation and real-time implementation, along with a summary and analysis of the system performance.
- Chapter 6 concludes the project by summarizing the work carried out and discusses future scope and possible enhancements to improve the system.

Chapter 2

Literature Survey

The literature survey presents a comprehensive study of various approaches used in Micromouse maze-solving systems, highlighting different techniques ranging from image-based mapping to sensor-driven navigation and algorithmic path planning. These approaches demonstrate how autonomous robots can perceive their env, build internal representations, and determine efficient paths toward a goal. The first approach introduces an image-processing-based method where the maze is analyzed using computer vision techniques and solved using the Breadth-First Search (BFS) algorithm. In this method, an external image of the maze is captured and processed to identify walls, paths, and key points such as the start and goal positions. The BFS algorithm is then applied to determine the shortest path. This approach reduces onboard hardware requirements and simplifies robot design. However, it is limited by its dependence on external image capture and lacks real-time adaptability, making it unsuitable for dynamic or unknown environments. The second approach focuses on the Flood-Fill algorithm as presented in the IEEE Bruins Micromouse project. This method assigns distance values to each cell in the maze starting from the goal and propagates these values throughout the grid. As the robot explores the maze, it updates its internal map and follows paths with decreasing distance values. This ensures that the robot eventually discovers the shortest path. The approach is highly efficient for grid-based mazes and supports dynamic updates as new walls are detected. The third study provides a detailed analysis of Micromouse system design, including mechanical structure, sensor systems, and various maze-solving algorithms such as Wall Following, Depth-First Search (DFS), and Flood-Fill. Wall-following is simple but inefficient in complex mazes, while DFS explores all possible paths without guaranteeing optimality. Among these, Flood-Fill is identified as the most effective algorithm due to its ability to compute the shortest path efficiently while supporting real-time decision-making. The fourth approach discusses the implementation of a maze-solving robot using Arduino Uno, ultrasonic sensors, and a combination of Wall-Following and Flood-Fill algorithms. In this hybrid method, the robot initially explores the maze using wall-following logic and later computes the shortest path using Flood-Fill. This combination improves navigation reliability and ensures that the robot can operate effectively even in partially known environments. However, the system is affected by sensor inaccuracies and has been tested only in small-scale environments.

The fifth study presents a detailed dissertation on Micromouse development, focusing on hardware integration and algorithmic implementation. It introduces

Lee's Algorithm, which is a variant of the Flood-Fill approach, for determining optimal paths. The study emphasizes the importance of sensor placement, calibration, and decision-making logic. It also highlights that successful maze solving depends not only on the algorithm but also on the proper integration of hardware components such as sensors, microcontrollers, and motor drivers. Overall, the literature indicates that while multiple algorithms exist for maze solving, Flood-Fill and its variants are the most effective for achieving optimal path planning in grid-based environments. The performance of a Micromouse system is strongly influenced by sensor accuracy, real-time mapping capability, and efficient control mechanisms. Based on these observations, the proposed system adopts a sensor-based approach using infrared sensors and the Flood-Fill algorithm. This enables real-time exploration, dynamic map construction, and adaptive path planning, resulting in efficient, accurate, and fully autonomous maze-solving behavior.

2.1 Maze Solving with Image-Based Mapping

[1] This paper is by **Mihai-Sebastian Mănase and Eva-H. Dulf** presents an advanced and innovative approach to **autonomous maze solving**.

The primary objective of the study is to design a **cost-effective robotic system** capable of navigating complex maze environments while minimizing **energy**, and system complexity. The proposed system leverages **image processing** and **artificial intelligence techniques** to extract meaningful information from a captured image of the maze. The approach involves multiple stages, including **image preprocessing**, **edge detection**, **depth estimation**, and **object recognition**. Advanced deep learning models such as **YOLOv8** are employed to detect key elements like the **start point**, **goal position**, and **obstacles**, while **neural networks** are used to enhance obstacle detection through depth analysis. These techniques enable the system to construct a detailed grid-based representation of the maze environment, which is essential for accurate path planning. Once the maze is represented in a structured format, the **Breadth-First Search (BFS)** algorithm is applied to compute the **shortest possible path** between the start and goal positions. The BFS algorithm ensures **optimal pathfinding** by exploring all possible paths level by level and selecting the minimum-distance route. The computed path is then transmitted to the robot, which executes navigation using a **cascade control system**. This includes both **velocity control** and **position control**, ensuring **precise movement**, and **accurate tracking** during navigation. A significant contribution of this work is the **reduction in hardware complexity**. Unlike traditional Micromouse systems that rely heavily on **onboard sensors** for real-time mapping, this approach utilizes **external image processing**, thereby reducing the need for multiple sensors. This results in **lower power consumption**, **reduced system cost**, and **simplified robot design**. The system demonstrates that high-performance maze solving can be achieved even with minimal hardware resources. Furthermore, the paper highlights the importance of preprocessing accuracy and image clarity in ensuring reliable maze representation. Any noise, distortion, or incorrect detection in the image processing stage can directly affect the accuracy of the computed path.

Experimental results show that the system achieves performance comparable to

sensor-based approaches, validating the effectiveness of the method. However, the system has certain limitations. It assumes the availability of a **clear overhead view** of the maze, which may not always be feasible in real-world scenarios, the approach is limited to **static environments**, as the path is precomputed based on a fixed image and does not adapt to changes such as moving obstacles or dynamic conditions.

The lack of **real-time adaptability** restricts its use in unpredictable env. In contrast, the system proposed in this project adopts a -based approach using **IR sensors** and the Flood-Fill algorithm ,enabling -time exploration, **dynamic** , and **adaptive decision-making**. While the image-based method offers advantages such as **simplicity** and **low cost**, the proposed approach provides greater **flexibility**, **autonomy**, and **robustness**, making it more suitable for practical Micromouse applications and real-world robotic navigation systems.

2.2 Bruins Micromouse Flood Fill Module

[2] This work presents a detailed study of the **Flood-Fill algorithm** for maze solving in Micromouse robots, as described in the IEEE Bruins Micromouse project. The approach focuses on efficient path planning in grid-based mazes, where the robot must navigate from a starting point to a goal cell using minimal time and distance. The Flood-Fill algorithm operates by assigning distance values to each cell in the maze, starting from the goal position and propagating outward. Each cell is given a value based on its distance from the goal, and the robot selects the neighboring cell with the lowest value to determine its movement. This method ensures that the robot always follows the shortest path once the maze is fully explored. The system relies on sensors to detect walls and update the internal representation of the maze dynamically. As the robot explores unknown regions, the distance values are recalculated to reflect newly discovered obstacles. This allows the robot to adapt to the environment in real time and refine its path accordingly. One of the key advantages of this approach is its efficiency and suitability for grid-based maze structures commonly used in Micromouse competitions. The algorithm is simple to implement and guarantees optimal pathfinding under accurate mapping conditions. However, the performance of the system heavily depends on reliable sensor input and accurate wall detection. Compared to simpler methods such as wall-following, the Flood-Fill algorithm provides a more intelligent and optimal solution by combining mapping and path planning. This approach forms the foundation of the proposed system in this project, *where the robot IR sensors and real-time computation to solve the maze efficiently.*

2.3 The Maze Problem Solved by Micromouse

[3] This paper titled “*The Maze Problem Solved by Micromouse*” by Sandeep Yadav, Kamal Kumar Verma, and Swetamadhab Mahanta presents a study on the design and implementation of an *autonomous Micromouse robot* capable of solving a maze using various *path-planning algorithms*.

The study emphasizes the importance of **decision-making algorithms** and **intelligent control** in robotic systems. A Micromouse is described as a miniature robot consisting of three major subsystems: the **drive system**, **sensor array**, and **control system**. The primary objective of the robot is to navigate from a predefined starting position to the **center of a maze** in the **shortest possible time**. The robot must operate in an **unknown environment**, requiring it to **detect walls**, **map the maze**, and make **real-time navigation decisions**. The paper discusses the **mechanical design aspects** of the robot, including the use of **lightweight materials** and **precision fabrication techniques** to improve efficiency and performance. It also highlights the importance of **sensor systems** in detecting walls and measuring distances accurately. Various sensors such as **IR sensors**, **LED-LDR combinations**, and **ultrasonic sensors** are analyzed based on their *range*, *reliability*, and *cost*. A major contribution of the paper is the detailed analysis of different maze-solving algorithms, including **wall-following**, **Depth-First Search (DFS)**, and **Flood-Fill**. The **wall-following algorithm** is simple and easy to implement but fails in certain maze configurations. **DFS** ensures that the robot eventually finds the goal but is inefficient as it explores all possible paths without guaranteeing the *shortest route*. The **Flood-Fill algorithm** is presented as the most efficient approach for maze solving. It assigns **distance values** to each cell in the maze based on their distance from the goal, allowing the robot to navigate by selecting the **cell with the lowest value**. The maze is represented as a **16×16 grid**, and distance values are propagated throughout the grid to determine the **optimal path**. This method significantly reduces the **time required** to reach the goal compared to other algorithms. In conclusion, the study demonstrates that while multiple algorithms exist for maze solving, the **Flood-Fill algorithm** provides the best balance between **efficiency**, **accuracy**, and **practicality**. This directly supports the approach used in the proposed system, where **Flood-Fill** is implemented along with **sensor-based mapping** for efficient and autonomous maze navigation.

2.4 Maze Solving Robot Using Arduino Uno

[4] This paper titled “*A Review on: Maze Solving Robot Using Arduino Uno*” presents a detailed study on the design and implementation of an **autonomous maze-solving robot** using **embedded systems** and decision-making.

The paper emphasizes the importance of **intelligent navigation in robotics** and highlights how *autonomous robots operate independently in unknown env.* The system is built using an **Arduino Uno microcontroller**, **ultrasonic sensors** for obstacle detection, and **motor drivers** for controlling movement. The robot is capable of detecting walls in three directions—**left**, **right**, and **front**—by measuring distances using ultrasonic sensors. Based on these inputs, the robot makes **real-time navigation decisions**. The paper discusses two major algorithms used for maze solving: the **Wall Following algorithm** and the **Flood-Fill algorithm**. The **wall-following approach** is used for basic navigation, where the robot follows either the left or right wall continuously. This method is *simple and easy to implement* but does not guarantee the **shortest path** and fails in *complex maze structures*. To overcome this limitation, the **Flood-Fill algorithm** is introduced to determine the

shortest path. In this method, the maze is divided into **grid cells**, and **distance values** are assigned starting from the **goal cell**. The algorithm propagates values to neighboring cells, and the robot navigates by moving from **higher-value cells to lower-value cells**, thereby reaching the goal efficiently.

The paper also highlights the **integration of both algorithms**, where the robot initially explores the maze using **wall-following logic** and later computes the **optimal path** using the Flood-Fill algorithm. This **hybrid approach** improves the **efficiency of navigation** and ensures reliable maze solving. In conclusion, the study demonstrates that combining **simple navigation techniques** with **advanced path-planning algorithms** significantly improves maze-solving performance. This approach closely aligns with the proposed system, which uses **sensor-navigation** along with the **Flood-Fill algorithm** for efficient and autonomous maze solving.

2.5 Design of a Maze-Solving Mouse

[5] This dissertation titled “*Maze Mouse*” presents a comprehensive study on the design and development of a **Micromouse robot**, focusing on the **integration of hardware components and maze-solving algorithms**. The project emphasizes the importance of combining **sensory systems, microcontroller-based control, and motor driver mechanisms** to achieve **autonomous navigation** in a maze environment. The Micromouse system is structured into three main subsystems: the **sensory system**, the **control system**, and the **drive system**. The sensory system primarily utilizes **infrared (IR) sensors** to detect walls and obstacles. These sensors operate based on *reflective principles*, allowing the robot to determine the presence and position of walls in **real time**. The control system is implemented using an **8051-based microcontroller**, which processes sensor inputs and executes **decision-making algorithms**. The drive system consists of **stepper motors**, enabling **precise control** over movement and direction. A key contribution of this work is the implementation of **Lee’s Algorithm** (also known as the **Bellman Flooding Algorithm**) for maze solving. The algorithm assigns **values to each cell** in the maze starting from the **target location** and propagates these values outward. The robot then selects the next move based on the **lowest value among neighboring cells**, ensuring an **optimal path** toward the goal. The algorithm also incorporates **dynamic updates** when new wall information is detected, improving **adaptability in unknown environments**. Sensors are positioned ahead of the robot’s center to allow effective **wall detection** and **steering correction**. The analysis shows that improper sensor placement can lead to **inaccurate navigation** and failure to reach the goal. In addition to algorithmic design, the dissertation provides a detailed explanation of **hardware implementation**, including **MC interfacing, motor driver circuits, and system integration**. The study also introduces a **decision matrix approach**, where sensor inputs are converted into **binary signals** and mapped to specific movement actions such as *forward motion, turning, or reversing*. In conclusion, this work provides a strong foundation for Micromouse system design by integrating **hardware and algorithmic approaches**. It demonstrates that efficient maze solving requires not only an **optimal algorithm** but also **precise sensor integration** and **reliable control systems**.

Chapter 3

Proposed System

3.1 System Overview

The proposed system is an autonomous Micromouse robot designed to navigate and solve a maze efficiently using sensor-based perception and intelligent path-planning algorithms. The primary objective of the system is to move from a predefined starting position to the goal location by determining the shortest possible path within the maze. The system operates using a combination of hardware and software components. Infrared (IR) sensors are used to detect walls and obstacles in real time, allowing the robot to understand its surroundings. The Arduino Nano microcontroller acts as the central processing unit, receiving sensor inputs, processing the data, and making navigation decisions. The motor driver (TB6612FNG) controls the movement of the motors based on the commands generated by the microcontroller. The navigation of the robot is based on the Flood-Fill algorithm, which enables efficient path planning in grid-based mazes. As the robot explores the maze, it continuously updates its internal representation by detecting walls and assigning distance values to each cell. Based on these values, the robot selects the optimal path toward the goal. To improve reliability and performance, a simulation of the maze-solving process is implemented using Python. This allows testing and validation of the algorithm in a virtual environment before deploying it on hardware. The integration of simulation and real-time implementation ensures navigation and efficient maze-solving. Overall,

the system demonstrates the application of autonomous decision-making, realtime mapping, and algorithmic optimization in robotics, providing a practical solution for intelligent maze navigation.

3.1.1 System Architecture

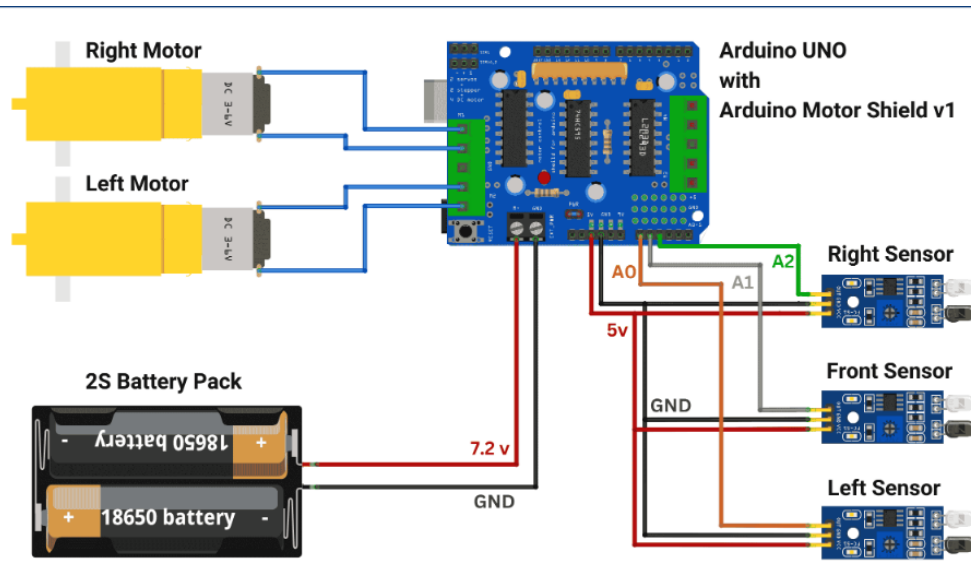


Figure 3.1: System Architecture of Micromouse Robot (Arduino Nano Based)

The system architecture of the proposed Micromouse robot consists of multiple interconnected hardware modules that work together to achieve autonomous maze navigation. The architecture is designed to support real-time sensing, intelligent decision-making, and precise motor control, enabling the robot to explore and solve the maze efficiently.

The system is organized into five main modules, each performing a specific function:

- Sensor Module:** The sensor module consists of three infrared (IR) sensors positioned at the left, front, and right sides of the robot. sensors are responsible for detecting walls and obstacles by measuring reflected infrared signals. Based on the sensor readings, the robot determines whether a path is open or blocked. Continuous sensing allows the robot to dynamically understand its surrounding and update its internal maze representation.
- Control Unit (Arduino Nano):** The Arduino Nano serves as the central processing unit of the system. It receives input signals from the IR sensors and processes them in real time. The microcontroller executes the Flood-Fill algorithm to determine the shortest path to the goal.

It also handles decision-making tasks such as selecting movement direction, updating maze data, and coordinating all system operations.

- Motor Driver Module (TB6612FNG):** The motor driver module acts as an interface between the Arduino Nano and the DC motors. Since the microcontroller cannot supply sufficient current to drive the motors directly, the TB6612FNG motor driver amplifies the control signals and regulates motor speed and direction. It enables precise control of forward movement, turning, and stopping actions.

- **Actuator Module (DC Motors):** The actuator module consists of two DC motors connected to the wheels of the robot. These motors convert electrical signals into mechanical motion, allowing the robot to move within the maze. By controlling the speed and direction of each motor independently, the robot can perform movements such as forward motion, left turn, right turn, and reverse.
- **Power Supply Module:** The system is powered using a battery pack (typically 2S Li-ion or 18650 batteries). The power supply module provides the necessary voltage and current to all components, including the Arduino Nano, sensors, motor driver, and motors. Proper voltage regulation ensures stable and reliable operation of the entire system.

The overall working of the system follows a continuous feedback loop. The IR sensors first detect the presence of walls and send corresponding signals to the Arduino Nano. The microcontroller processes these inputs and updates the internal map of the maze. Using the Flood-Fill algorithm, it calculates the optimal path and determines the next movement. Based on this decision, control signals are sent to the motor driver module, which drives the motors accordingly. The robot then moves to the next cell and repeats the sensing and decision-making process. This loop continues until the robot successfully reaches the goal position. This architecture ensures efficient coordination between sensing, processing, and actuation, enabling accurate navigation and real-time adaptation in unknown maze environments.

3.1.2 Working Principle

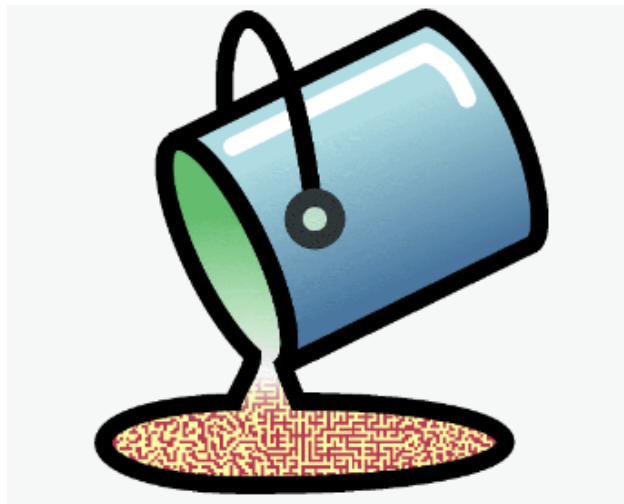


Figure 3.2: Flooding the Maze with numbers.

The working principle of the proposed Micromouse robot is based on continuous sensing, mapping, and intelligent path planning using the Flood-Fill algorithm. The system operates in a closed-loop manner where the robot repeatedly senses its environment, updates its internal map, and makes movement decisions.

Initially, the robot starts at a predefined position in the maze. The infrared (IR) sensors detect walls in three directions: left, front, and right. Based on these inputs, the Arduino Nano updates the internal representation of the maze, which is modeled as a grid of cells.

3.1.3 Flood-Fill Algorithm (Mathematical Representation)

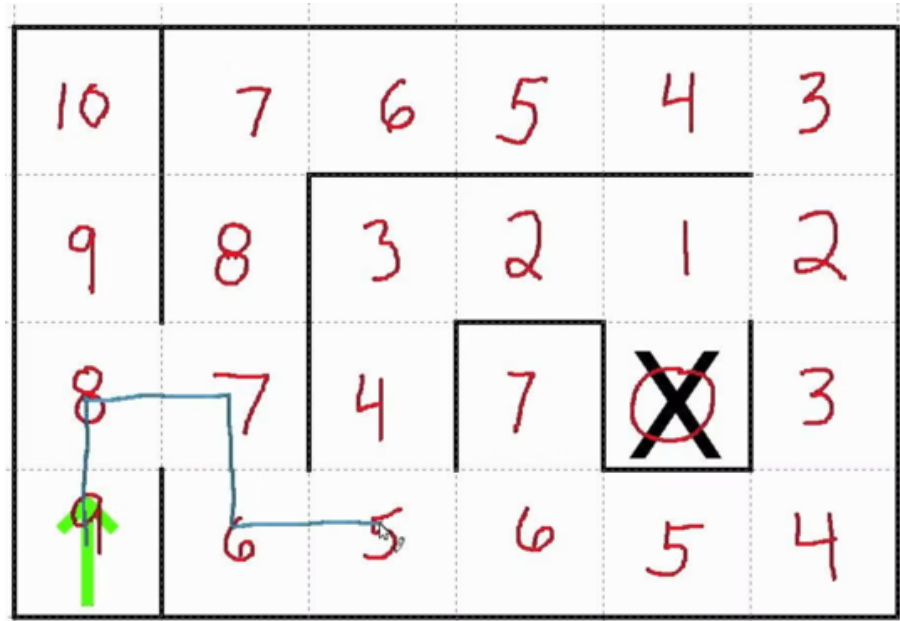


Figure 3.3: Flood-Fill Distance Map and Path Selection

The Flood-Fill algorithm assigns a value to each cell in the maze representing its distance from the goal. The goal cell is assigned a value of 0, and values increase as we move away from the goal.

$$D(x, y) = 1 + \min(D(x + 1, y), D(x - 1, y), D(x, y + 1), D(x, y - 1))$$

where $D(x, y)$ represents the distance of a cell from the goal, considering only accessible neighboring cells.

3.1.4 Example Based on Figure

As shown in Figure 4.3, each cell in the maze is assigned a numerical value representing its distance from the goal (marked with an “X”). The values decrease as the robot gets closer to the goal and increase as it moves away from it.

The robot starts from the bottom-left corner of the maze. At each step, it evaluates its neighboring cells and selects the cell with the lowest value. For example:

- From the starting position (value 9), the robot moves to the adjacent cell with value 8.
- It continues moving through cells with values 7, 6, and 5, following the decreasing sequence.

- The path avoids walls and obstacles, which block certain directions and force the robot to choose alternative routes.
- This process continues until the robot reaches the goal cell (value 0 or target position).

The blue path shown in the figure represents the actual path taken by the robot, which follows the gradient of decreasing values. The numbers in the grid act as a guide, ensuring that the robot always moves toward the shortest path.

3.1.5 Working Steps

1. Assign the goal cell with value 0
2. Propagate distance values to all reachable cells
3. Detect walls using IR sensors and update the maze map
4. At each step, compare neighboring cell values
5. Move to the cell with the smallest value
6. Repeat until the goal is reached

After selecting the next move, the Arduino Nano sends control signals to the motor driver (TB6612FNG), which drives the motors accordingly. The robot updates its path if new obstacles are detected, ensuring adaptability in unknown environments. Thus, the Flood-Fill algorithm enables efficient and optimal navigation by guiding the robot through a sequence of decreasing distance values until it reaches the goal.

3.1.6 Algorithm Design

The maze-solving process in the proposed system is implemented using the Flood-Fill algorithm, which is an efficient path-planning technique widely used in Micromouse robots. The algorithm determines the shortest path from the start position to the goal by assigning distance values to each cell in a grid-based maze representation. The maze is modeled as a two-dimensional grid of size $N \times N$, where each cell represents a possible position of the robot. The Flood-Fill algorithm works by propagating distance values from the goal cell to all other reachable cells, effectively creating a distance map. This map guides the robot in selecting the optimal path during navigation.

3.1.7 Algorithm Description

The algorithm consists of two main phases:

- **Distance Mapping Phase:** In this phase, the algorithm assigns distance values to each cell starting from the goal position. A queue-based approach (similar to Breadth-First Search) is used to propagate values to neighboring cells. Each cell is assigned a value equal to one plus the minimum value of its accessible neighbors.

- **Navigation Phase:** In this phase, the robot uses the computed distance values to move toward the goal. At each step, it selects the neighboring cell with the smallest value, ensuring movement along the shortest path.

3.1.8 Pseudo Code

```
Initialize all cells with a large value (infinity)
Set goal cell value = 0

Create a queue and add goal cell

# Distance Mapping Phase
while queue is not empty:
    current = dequeue()

    for each neighbor of current (up, down, left, right):
        if neighbor is not a wall:
            if value(neighbor) > value(current) + 1:
                value(neighbor) = value(current) + 1
                enqueue(neighbor)

# Navigation Phase
current = start cell

while current != goal:
    choose neighbor with minimum value
    move to that cell
    update current position
```

3.1.9 Working Explanation

Initially, all cells are assigned a large value representing infinity. The goal cell is assigned a value of zero. Using a queue, the algorithm explores neighboring cells and assigns increasing values based on their distance from the goal. As the algorithm progresses, it ensures that each cell holds the minimum distance required to reach the goal. This process continues until all reachable cells are updated. During navigation, the robot uses this distance map to determine its movement. At each step, it evaluates adjacent cells and moves to the one with the smallest value. This guarantees that the robot follows the shortest path. If new walls are detected during exploration, the distance values are recalculated dynamically, allowing the robot to adapt to unknown or partially explored environments.

3.1.10 Time Complexity

The Flood-Fill algorithm processes each cell in the grid at most once. For a maze of size $N \times N$, the total number of cells is N^2 .

$$TimeComplexity = O(N^2)$$

This is because:

- Each cell is visited once
- Each visit involves checking up to 4 neighbors

Thus, the overall time complexity grows quadratically with the size of the maze.

3.1.11 Space Complexity

The algorithm requires memory to store:

- Distance values for each cell in the grid
- A queue for processing cells

$$SpaceComplexity = O(N^2)$$

This is because the storage requirement is directly proportional to the number of cells in the maze.

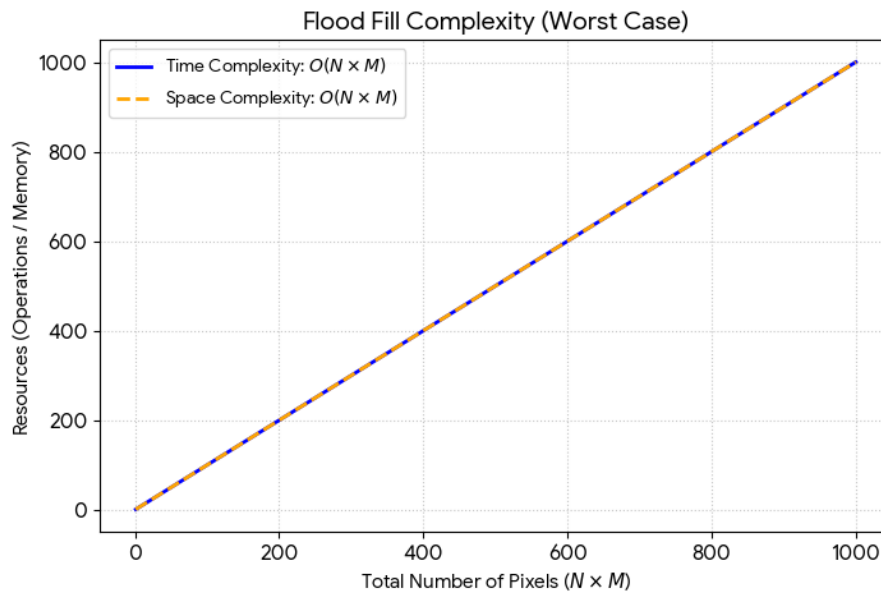


Figure 3.4: Time and Space Complexity

The graph represents the quadratic growth of the Flood-Fill algorithm. As the number of cells increases, the number of operations increases proportionally to N^2 . The graph clearly shows that as the input size increases, the number of operations increases quadratically. This makes the Flood-Fill algorithm efficient for moderate-sized mazes while still guaranteeing the shortest path.

3.2 Simulation of Maze

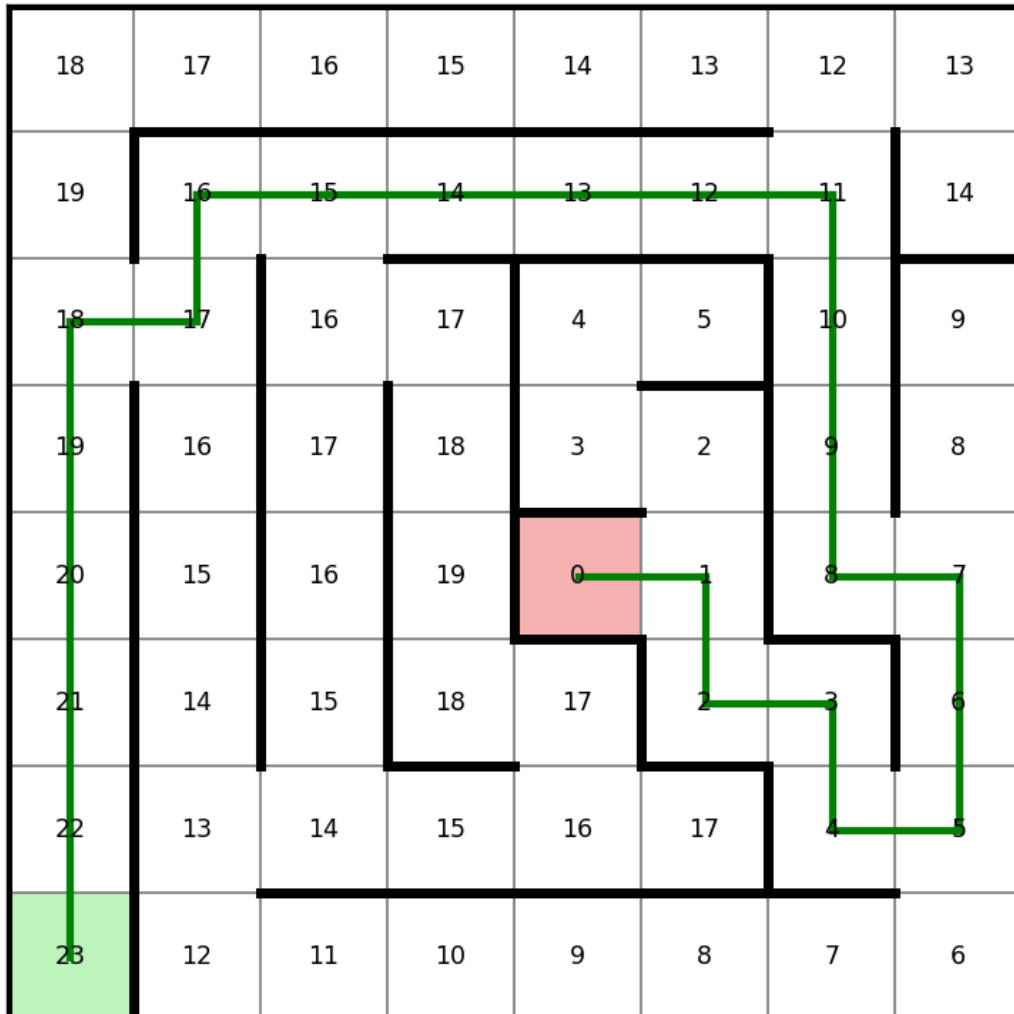


Figure 3.5: Simulation of the maze and the working of the algorithm

To validate the performance of the proposed Micromouse system, a simulation of maze generation and solving was implemented using Python. The simulation allows testing of the Flood-Fill algorithm in a virtual environment before deploying it on the physical robot. This helps in debugging, analyzing performance, and verifying correctness of the path-planning logic.

3.2.1 Maze Construction

A custom maze is created interactively using a grid-based interface. The maze is represented as a three-dimensional array, where each cell stores information about the presence or absence of walls in four directions: left, right, top, and bottom.

- The maze size is defined as 8×8 .
- Each cell contains four wall indicators.

- Walls can be added or removed dynamically using mouse clicks.

The system ensures consistency by synchronizing walls between adjacent cells. For example, if a wall is added to the right side of a cell, the corresponding left wall of the neighboring cell is also updated.

Once the maze is constructed, the configuration is saved in a file (`maze_8x8.py`) for later use in the solving phase.

3.2.2 Maze Representation

The maze is stored as a structured dataset containing wall information for each cell. A dictionary-based representation is used to efficiently access wall data during path computation.

- Each cell is identified by its coordinates (y, x) .
- Wall information is stored as binary values (0 for no wall, 1 for wall).

This representation allows quick checking of valid movements between neighboring cells.

3.2.3 Flood-Fill Implementation

The Flood-Fill algorithm is implemented using a queue-based approach similar to Breadth-First Search (BFS). The goal cell is initialized with a value of zero, and values are propagated outward to all reachable cells.

- A distance matrix is initialized with infinity.
- The goal cell is assigned a value of 0.
- A queue is used to propagate values to neighboring cells.
- Only valid neighbors (without walls) are considered.

The algorithm ensures that each cell contains the minimum number of steps required to reach the goal.

3.2.4 Path Extraction

Once the flood-fill values are computed, the shortest path is extracted starting from the initial position. The robot selects the neighboring cell with the minimum value at each step.

- The start position is defined as $(7, 0)$.
- The goal position is set at the center of the maze.
- At each step, the next move is chosen based on the lowest distance value.

This guarantees that the robot follows the optimal path toward the goal.

3.2.5 Visualization

The simulation includes a graphical visualization of the maze using Matplotlib. The following elements are displayed:

- Grid structure of the maze
- Walls represented as thick black lines
- Distance values in each cell
- Start position highlighted in green
- Goal position highlighted in red
- Final path drawn as a green line

The visualization helps in understanding how the algorithm propagates values and selects the shortest path. The simulation results demonstrate that the Flood-Fill algorithm successfully computes the shortest path in the maze. The robot consistently follows a sequence of decreasing distance values until it reaches the goal.

Key observations include:

- The algorithm efficiently handles different maze configurations.
- Path recalculation is possible when new walls are introduced.
- The simulation closely replicates real-world robot behavior.

The simulation confirms the effectiveness of the Flood-Fill algorithm for maze solving. It provides a reliable platform for testing and validating the navigation logic before implementing it on hardware. This reduces errors and improves overall system performance.

3.2.6 Mapping & Navigation



Figure 3.6: Mapping and Navigation

Mapping and navigation are the core components of the proposed Micromouse system, enabling the robot to explore an unknown maze, construct an internal representation, and determine the optimal path to the goal.

3.2.7 Maze Mapping

The maze is represented as a grid of size $N \times N$, where each cell corresponds to a possible position of the robot. Each cell stores information about the presence or absence of walls in four directions: left, right, top, and bottom. As the robot moves through the maze, it uses infrared (IR) sensors to detect walls in real time. Based on the sensor readings, the robot updates its internal map by marking walls and open paths. This process is performed incrementally, meaning the robot gradually builds the complete maze structure as it explores.

The mapping process includes:

- Detecting walls using IR sensors
- Updating wall information in the current cell
- Synchronizing wall data with neighboring cells
- Storing the map in a structured format (grid or matrix)

This internal representation allows the robot to keep track of explored and unexplored regions of the maze.

3.3 Distance Mapping using Flood-Fill

Once the maze information is available, the Flood-Fill algorithm is used to assign distance values to each cell. The goal cell is assigned a value of 0, and values increase as we move away from the goal. Each cell value represents the minimum number of steps required to reach the goal. These values are updated dynamically as new walls are discovered during exploration. The distance mapping helps the robot in making efficient navigation decisions by providing a numerical guide toward the goal.

3.3.1 Navigation Strategy

The navigation of the robot is based on a greedy approach guided by the Flood-Fill values. At each step, the robot evaluates all accessible neighboring cells and selects the one with the smallest distance value.

The navigation process follows these steps:

1. Sense the environment and detect walls
2. Update the maze map
3. Recalculate Flood-Fill values if needed
4. Identify all valid neighboring cells
5. Select the neighbor with the minimum distance value
6. Move the robot to the selected cell

This ensures that the robot always moves toward the goal using the shortest possible path.

3.3.2 Dynamic Adaptation

In unknown maze environments, the robot may encounter new walls that were not previously mapped. In such cases, the Flood-Fill values are recalculated to reflect the updated maze configuration.

This dynamic updating allows the robot to:

- Avoid incorrect paths
- Adapt to newly discovered obstacles
- Maintain optimal navigation throughout exploration

3.3.3 Path Optimization

After reaching the goal, the robot can use the fully mapped maze to compute the fastest path. This path may differ from the initial exploration path, as it eliminates unnecessary movements and optimizes turns. The optimized path is then used for faster traversal during subsequent runs.

The combination of real-time mapping and intelligent navigation enables the robot to efficiently explore and solve the maze. The integration of sensor data with the Flood-Fill algorithm ensures accurate path planning and adaptability in dynamic environments.

3.3.4 Flowchart

The flowchart represents the complete step-by-step working of the Micromouse robot based on the Flood-Fill algorithm. It provides a clear visualization of how the robot operates in a continuous loop of sensing, processing, and movement until it successfully reaches the goal position. The flowchart helps in understanding the logical sequence of operations and the decision-making process involved in maze navigation.

The process begins with the initialization stage, where the robot is placed at a predefined starting position in the maze. At this stage all cells are assigned default values. The goal position is also defined, which serves as the reference point for the Flood-Fill algorithm. This initialization ensures that the robot has a structured framework to begin its exploration. Once initialized, the robot enters a continuous execution loop. The first step in this loop is sensing the environment using infrared (IR) sensors. These sensors detect the presence or absence of walls in three directions: left, front, and right. The sensor readings are converted into digital signals and sent to the microcontroller for processing. After sensing, the robot updates its internal map of the maze. The detected walls are recorded in the grid representation, and the information is synchronized with neighboring cells to maintain consistency. This step is crucial because accurate mapping directly affects the efficiency of navigation. Following the map update, the Flood-Fill algorithm is applied. In this step, distance values are assigned to each cell based on their proximity to the goal. The algorithm propagates values from the goal cell outward, ensuring that each cell stores the minimum number of steps required to reach the goal. If new walls are detected, the distance values are recalculated dynamically to reflect the updated maze structure. Once the distance map is computed, the robot proceeds to the decision-making stage. At this point, the robot evaluates all accessible neighboring cells and selects the one with the smallest distance value. This ensures that the robot always moves in the direction of the shortest path toward the goal. After selecting the optimal direction, the robot generates control signals to drive the motors and perform the required movement. The robot then physically moves to the next cell and updates its current position. After each movement, the system checks whether the goal has been reached. If the goal is not reached, the robot repeats the cycle of sensing, mapping, and decision-making. This loop continues until the robot successfully reaches the goal cell. If new obstacles or walls are encountered during movement, the system adapts by recalculating the Flood-Fill values, ensuring that the robot always follows the most efficient path. This dynamic updating capability allows the robot to operate effectively in unknown or partially explored environments. Overall, the flowchart represents a closed-loop control system where sensing, computation, and action are continuously performed. This iterative process enables the Micromouse robot to efficiently explore the maze, adapt to changes, and determine the shortest path to the goal in a reliable and

autonomous manner.

Workflow

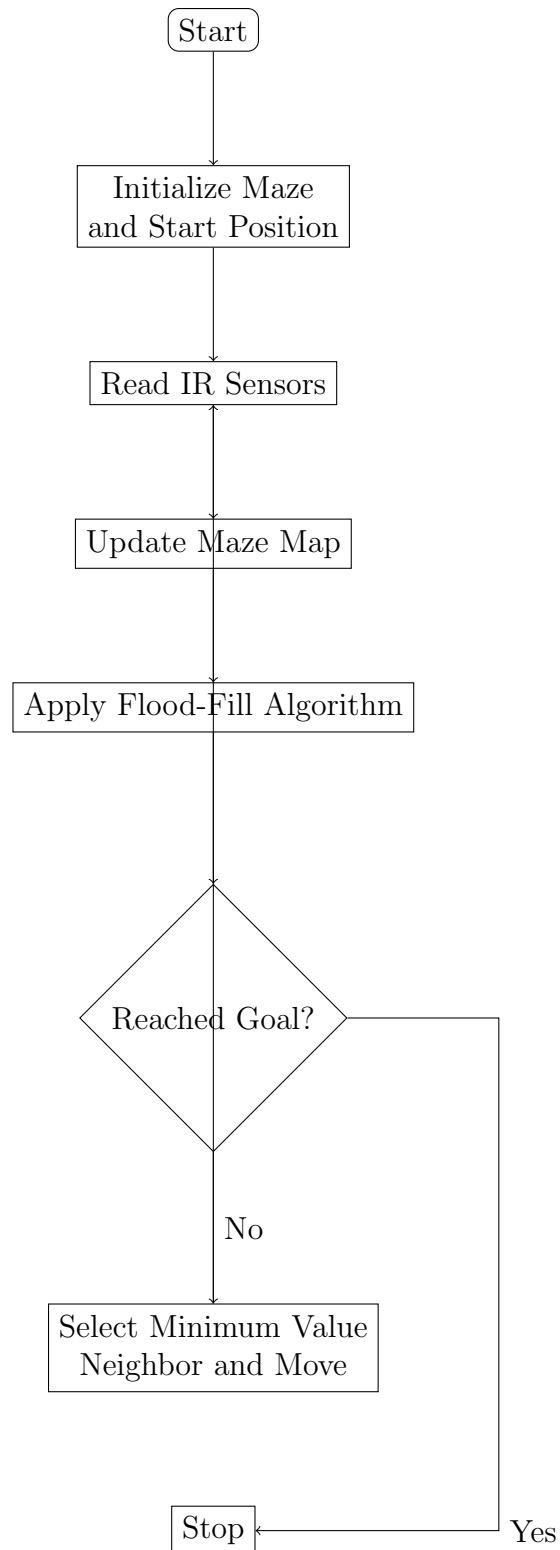


Figure 3.7: Workflow of Maze Solving Micro-Mouse

3.3.5 Advantages

The proposed Micromouse system offers several advantages in terms of efficiency, accuracy, and adaptability. The integration of sensor-based mapping with the F-F algorithm enables the robot to perform autonomous navigation in a structured and optimized manner.

- **Shortest Path Guarantee:** The use of the Flood-Fill algorithm ensures that the robot always finds the shortest path to the goal once the maze is sufficiently explored. This significantly improves navigation efficiency compared to simpler methods like wall-following.
- **Real-Time Mapping:** The system continuously updates its internal map using IR sensor data. This allows the robot to operate effectively in unknown environments and adapt to newly discovered walls during exploration.
- **Dynamic Path Recalculation:** If the robot encounters unexpected obstacles or new walls, the Flood-Fill algorithm recalculates the distance values dynamic. This ensures that the robot always follows the most optimal path.
- **Efficient Decision-Making:** The robot makes decisions based on numerical distance values assigned to each cell. This simplifies the navigation logic and reduces computational complexity.
- **Low Hardware Complexity:** The system uses simple and cost-effective components such as Arduino Nano, IR sensors, and a motor driver. This makes the design affordable and easy to implement.
- **Modular Design:** The architecture is modular, allowing individual component such as sensors, control unit, and motors to be easily modified or upgraded without affecting the overall system.
- **Simulation Support:** The use of Python-based simulation allows testing and validation of the algorithm before hardware implementation. This reduces development time and minimizes errors.
- **Scalability:** The algorithm and system design can be extended to larger maze sizes without significant changes, making it suitable for more complex.
- **Autonomous Operation:** The robot operates without human intervention, demonstrating the principles of autonomous systems and intelligent reasoning.
- **Application in Real-World Systems:** The concepts used in this project, such as path planning and real-time mapping, are applicable in real-world domains like autonomous vehicles, robotics, and navigation systems.

Chapter 4

Tools & Technologies Used

This chapter presents a comprehensive description of the hardware and software tools utilized in the development of the proposed Micromouse system. The selection of these components has been carefully made by considering several important factors such as performance, efficiency, reliability, cost-effectiveness, and suitability for real-time autonomous navigation. Each component plays a vital role in ensuring the smooth functioning of the system and contributes to the overall effectiveness of the robot. The hardware components are responsible for sensing the environment, processing inputs, and executing physical movements, while the software tools provide the necessary platform for implementing control logic, simulation, and algorithmic processing. The integration of these hardware and software elements enables the robot to operate as an intelligent autonomous system capable of interacting with its surroundings. Overall, the combination of carefully selected tools and technologies provides a robust foundation for the Micromouse robot, enabling efficient sensing, accurate decision-making, and reliable navigation within the maze environment.

4.1 Hardware Components

4.1.1 Arduino Nano

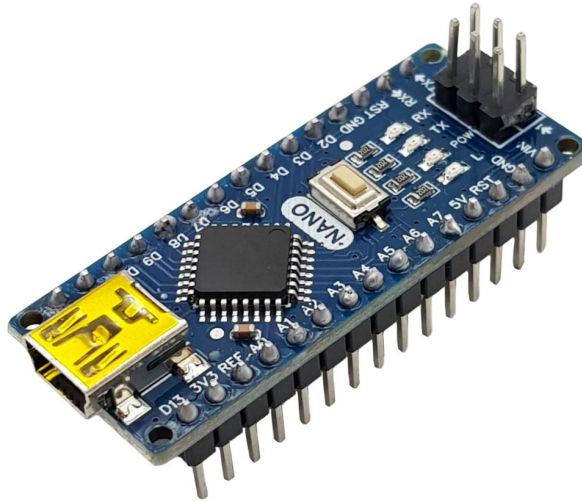


Figure 4.1: Microcontroller Board

The Arduino Nano serves as the central control unit of the Micromouse system and acts as the brain of the robot. It is based on the ATmega328P microcontroller, which is capable of executing instructions, processing sensor data, and controlling multiple hardware components simultaneously. The microcontroller is responsible for handling all decision-making processes and coordinating the overall operation of the system. The Arduino Nano continuously receives input signals from the infrared (IR) sensors, which provide information about the presence of walls and obstacles in the maze. Based on these inputs, the microcontroller updates the internal representation of the maze and executes the Flood-Fill algorithm to determine the optimal path toward the goal. It then generates appropriate control signals to the motor driver, which in turn controls the movement of the DC motors. The board provides multiple digital and analog input/output pins, allowing easy interfacing with sensors, motor drivers, and other peripherals. The digital pins are used for reading sensor inputs and sending control signals, while PWM-enabled pins are used for controlling motor speed. The analog pins can be used for precise sensor readings if required. The Arduino Nano operates at a clock frequency of 16 MHz, which is sufficient for real-time processing in small robotic systems. It also includes built-in communication interfaces such as UART, SPI, and I2C, enabling communication with other devices if needed. One of the major advantages of the Arduino Nano is its compact size and lightweight design, making it ideal for space-constrained applications like Micromouse robots. It is also energy-efficient, which is essential for battery-powered systems. The Arduino Nano is programmed using the Arduino Integrated Development Environment (IDE), which supports C/C++ programming.

The simplicity of the IDE, along with the availability of extensive libraries and community support, makes it easy to develop, test, and debug the system efficiently.

4.1.2 Infrared (IR) Sensors



Figure 4.2: Sensor Module

The system uses three infrared (IR) sensors positioned at the left, front, and right sides of the robot to enable effective environment sensing. These sensors play a crucial role in detecting walls and obstacles within the maze, allowing the robot to make real-time navigation decisions. IR sensors operate based on the principle of infrared reflection. Each sensor consists of an infrared emitter and a receiver. The emitter continuously transmits infrared light, and when this light encounters an object such as a wall, it is reflected back toward the receiver. The receiver detects this reflected signal, and based on its intensity, the system determines the presence and proximity of obstacles. The output of the IR sensor is typically in the form of an analog or digital signal, which is sent to the Arduino Nano for processing. The microcontroller interprets these signals to identify whether a wall is present in a particular direction. This information is then used to update the internal map of the maze and guide the robot's movement. The use of three IR sensors allows the robot to simultaneously detect obstacles in multiple directions—left, front, and right. This multi-directional sensing capability improves the robot's awareness of its surroundings and enables more accurate decision-making during navigation. Real-time feedback from the sensors ensures that the robot can continuously monitor changes in the environment. This is especially important in unknown maze conditions, where the robot must dynamically update its path based on newly detected walls. The performance of IR sensors depends on factors such as surface reflectivity, distance from the object, and ambient lighting conditions. Therefore, proper calibration of the sensors is necessary to achieve accurate and reliable detection. Overall, the

integration of IR sensors enables efficient obstacle detection, accurate mapping, and reliable navigation, making them an essential component of the Micromouse system.

4.1.3 Motor Driver (TB6612FNG)

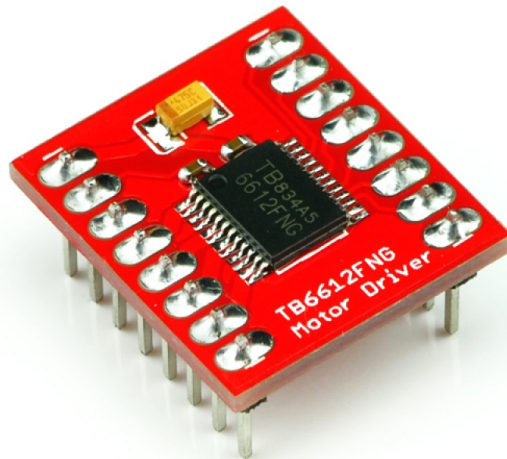


Figure 4.3: Driver Module

The TB6612FNG motor driver is used to control the DC motors of the Micromouse robot and acts as a crucial interface between the Arduino Nano and the motors. Since the Arduino Nano operates at low current levels and cannot supply sufficient power to drive motors directly, the motor driver is used to amplify control signals and provide the required current and voltage to the motors.

The TB6612FNG is a dual-channel motor driver, which means it can independently control two DC motors. This enables precise control over the movement of the robot, including forward motion, backward motion, turning, and stopping. Each motor can be controlled separately, allowing differential steering, where varying the speed of each motor results in smooth and accurate turns. The motor driver operates based on the H-bridge principle, which allows the direction of current flow through the motor to be reversed. By controlling the input signals, the system can change the polarity of the voltage applied to the motor, thereby controlling its direction of rotation. In addition to direction control, the TB6612FNG supports Pulse Width Modulation (PWM) for speed control. By varying the duty cycle of the PWM signal, the effective voltage supplied to the motor is adjusted, enabling smooth acceleration and deceleration. This is essential for achieving stable and precise navigation in the maze. Compared to traditional motor drivers such as the L298N, the TB6612FNG offers higher efficiency, lower power loss, and reduced heat generation. This makes it more suitable for compact and battery-powered robotic systems. Its small size and lightweight design further enhance its compatibility with Micromouse robots. The driver also includes built-in protection features such

as thermal shutdown and overcurrent protection, which improve the safety and reliability of the system. Overall, the TB6612FNG motor driver plays a critical role in ensuring efficient motor control, smooth movement, and accurate navigation of the robot.

4.1.4 DC Motors

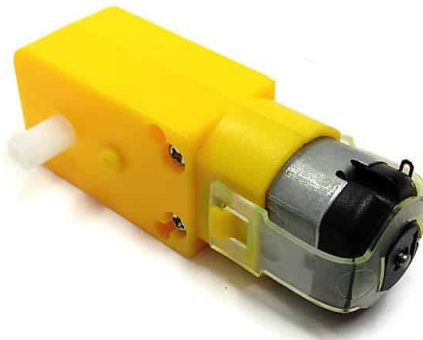


Figure 4.4: DC Motor Used in the Micromouse Robot

DC motors are used as the primary actuators in the Micromouse robot, playing a vital role in enabling movement and navigation within the maze. These motors convert electrical energy into mechanical motion through the interaction of magnetic fields, allowing the robot to move in different directions. The motors are directly connected to the wheels, and their rotation facilitates forward movement, backward movement, and directional turning. The operation of DC motors is based on the principle of electromagnetic induction, where the flow of current through the motor windings generates a magnetic field that produces rotational motion. This simple yet effective mechanism makes DC motors highly suitable for robotic applications that require continuous and controlled movement. The speed and direction of the DC motors are controlled using the TB6612FNG motor driver, which receives control signals from the Arduino Nano. By adjusting the voltage supplied to the motors and controlling the polarity of the current, the system can regulate both speed and direction. Additionally, Pulse Width Modulation (PWM) is used to achieve fine control over motor speed, enabling smooth acceleration and deceleration. Precise motor control is essential for accurate navigation in the maze. The ability to perform smooth turns and maintain consistent speed ensures that the robot can align correctly with maze paths and avoid collisions with walls. The coordination between the two motors allows the robot to execute differential steering, where varying the speed of each motor results in controlled turning. The performance of

the DC motors directly affects the overall efficiency and reliability of the system. Key parameters such as speed, torque, and response time determine how effectively the robot can navigate the maze. Adequate torque is required to overcome friction and ensure stable movement, while fast response time allows quick adjustments during navigation. In addition, proper calibration of motor speed is necessary to maintain synchronization between both wheels. Any imbalance can lead to deviation from the intended path. Therefore, careful control and tuning of the motors are crucial for achieving accurate and efficient maze-solving performance.

Robot Chassis



Figure 4.5: Robot Car Chassis Structure

The robot chassis serves as the structural foundation of the Micromouse system and provides a stable platform for integrating all hardware components. It supports and securely holds essential elements such as the Arduino Nano, infrared sensors, motor driver, batteries, and DC motors. The design of the chassis plays a crucial role in ensuring that all components are properly aligned and protected during operation. A well-designed chassis ensures accurate alignment of the wheels, which is essential for maintaining straight-line motion and executing precise turns within the maze. It contributes significantly to the stability of the robot by minimizing wobbling and unwanted movements.

Additionally, the chassis helps in reducing mechanical vibration that may arise due to motor operation, thereby improving the overall performance and reliability of the system. Proper placement of components on the chassis is critical for achieving uniform weight distribution. Balanced weight ensures that the robot maintains proper traction with the surface, preventing slippage and improving control during movement. This is particularly important when the robot performs turns or navigates narrow paths in the maze. The chassis also influences the size, speed of the robot. A compact and lightweight chassis allows the robot to move efficiently within the

limited space of the maze and enables quicker directional changes. This improves the robot's ability to navigate sharp corners and optimize its path.

Furthermore, the material and build quality of the chassis affect the durability and longevity of the system. A strong yet lightweight chassis enhances performance while ensuring that the robot can withstand repeated use during testing. Overall, the chassis is a critical component that directly impacts the robot's stability, efficiency, and navigation accuracy.

4.1.5 Power Supply (2 AA Rechargeable Batteries)



Figure 4.6: Power Module

The system is powered using two AA rechargeable batteries, which act as the primary energy source for the Micromouse robot. These batteries provide the necessary voltage and current required for the proper operation of all electronic components, including the Arduino Nano, infrared sensors, motor driver, and DC motors. The use of rechargeable batteries ensures cost-effectiveness and environmental sustainability, as they can be reused multiple times and reduce the need for frequent replacements.

This makes them suitable for repeated testing and experimentation during the development phase of the project. Despite their compact size, AA batteries are capable of delivering sufficient power for low-power robotic systems. They offer a good balance between size, weight, and energy capacity, making them ideal for compact robots like Micromouse, where space and weight are critical constraints. Proper power management is essential for the reliable functioning of the system. Voltage fluctuations or insufficient current supply can lead to unstable behavior, such as incorrect sensor readings or inconsistent motor performance. Therefore, ensuring secure connections and efficient power distribution is important for maintaining stable operation. In addition, the arrangement and placement of the batteries on the robot chassis must be carefully designed to maintain proper weight balance. This helps in improving stability, reducing vibrations, and ensuring smooth navigation within the maze.

4.1.6 Jumper Wires

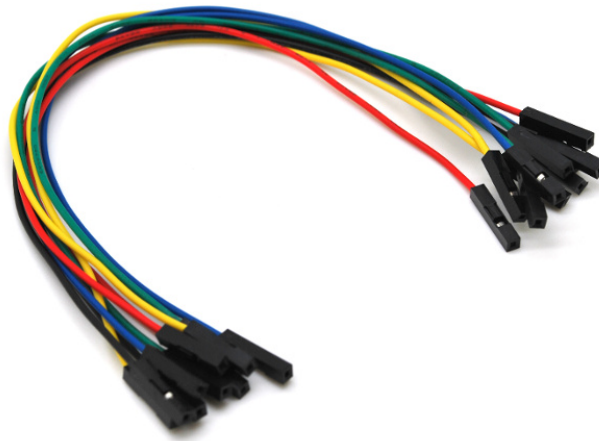


Figure 4.7: wires(F-F)

Jumper wires are used to establish electrical connections between various components of the system, including the Arduino Nano, IR sensors, motor driver, and power supply. These wires provide flexibility during prototyping and allow easy modification of the circuit without the need for permanent soldering. They are especially useful during the development phase, where frequent changes in connections may be required for testing and debugging. Jumper wires enable quick assembly and reconfiguration of the circuit, making the overall design process more efficient and less time-consuming. They play a crucial role in ensuring proper signal transmission and maintaining reliable connectivity between components. High-quality jumper wires help in reducing signal loss and electrical noise, which is important for accurate sensor readings and stable motor control.

4.2 Software Tools

4.2.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is used for writing and uploading code to the Arduino Nano. It provides a user-friendly interface and supports a wide range of libraries for sensor interfacing and motor control. The IDE simplifies the development process and allows quick testing and debugging of the embedded program.

4.2.2 Embedded Programming using C/C++ (.ino)

The control logic of the Micromouse robot is implemented using C/C++ in the form of an Arduino sketch (.ino file). The program handles various tasks such as

reading sensor inputs, updating the maze map, executing the Flood-Fill algorithm, and controlling motor movements. C/C++ is used due to its efficiency, speed, and suitability for embedded systems. It ensures real-time processing and quick response to sensor inputs, which is essential for autonomous navigation.

4.2.3 Python (Simulation)

Python is used for simulating the maze-solving process before implementing it on hardware. The simulation helps in verifying the correctness of the Flood-Fill algorithm and analyzing the robot's behavior in different maze configurations. This reduces the risk of errors during hardware implementation and allows testing of various scenarios without physical constraints.

Libraries Used :

- **NumPy:** Used for efficient handling of arrays and grid-based data structures.
- **Matplotlib:** Used for visualizing the maze, flood-fill values, and the final path.
- **Collections (deque):** Used for implementing queue operations required for the Flood-Fill algorithm.

4.2.4 Flood-Fill Algorithm

The Flood-Fill algorithm is the core technique used for path planning in the system. It is a grid-based algorithm that assigns distance values to each cell in the maze, representing the minimum number of steps required to reach the goal from that cell. The goal cell is initialized with a value of zero, and values are propagated outward to all other cells based on their relative distance. The algorithm operates by repeatedly updating each cell's value using the minimum value of its accessible neighboring cells. This process continues until all cells contain consistent distance values. During navigation, the robot selects the neighboring cell with the lowest distance value, ensuring that it always moves along the shortest path toward the goal. The algorithm is computationally efficient and guarantees an optimal path once the maze is fully explored.

4.2.5 Grid-Based Mapping

The maze environment is represented using a grid-based structure, where each cell corresponds to a specific location in the maze. Each cell stores essential information such as the presence of walls in four directions (north, east, south, and west) and the distance value assigned by the Flood-Fill algorithm. This structured representation simplifies the implementation of path-planning algorithms, as it allows easy access to neighboring cells and efficient storage of maze information. The grid model also enables systematic exploration of the maze, where the robot can track visited and unvisited cells. Additionally, grid-based mapping supports dynamic updates, allowing the robot to modify its internal representation as new walls are discovered. This ensures that the map remains accurate and reflects the real environment, which is crucial for reliable navigation.

4.2.6 Sensor-Based Navigation

The robot relies on real-time sensor inputs to perceive its environment and make navigation decisions. Infrared (IR) sensors are used to detect the presence of walls and obstacles by measuring reflected signals. These sensors provide continuous feedback to the microcontroller, enabling the robot to identify open paths and blocked directions. Based on the sensor readings, the robot updates its internal map by marking the detected walls. This information is then used in conjunction with the Flood-Fill algorithm to determine the optimal movement direction. The integration of sensing and decision-making allows the robot to operate autonomously without prior knowledge of the maze. Sensor-based navigation also enables the robot to adapt dynamically to changes in the environment. As new walls are detected, the robot recalculates the distance values and adjusts its path accordingly. This ensures robust and flexible navigation, even in complex and unknown maze configurations. Overall, the combination of real-time sensing, grid-based mapping, and intelligent path planning enables efficient and accurate maze solving by the Micromouse robot.

4.3 Summary

The combination of hardware and software tools used in this project enables the successful implementation of an autonomous Micromouse system. Each component plays a specific and crucial role in ensuring the overall functionality and efficiency of the system. The Arduino Nano acts as the central control unit, managing data processing and decision-making, while the infrared sensors provide accurate and continuous environmental feedback for obstacle detection. The integration of the TB6612FNG motor driver and DC motors enables precise control of movement, allowing the robot to navigate smoothly through the maze. The use of a well-designed robot chassis ensures structural stability and proper alignment, which further enhance navigation accuracy. Additionally, the use of rechargeable batteries provides a reliable and portable power source, making the system suitable for real-time operation. From a software perspective, the implementation of the Flood-Fill algorithm ensures optimal path planning by guiding the robot toward the shortest path. The use of embedded programming in C/C++ enables real-time control and fast response to sensor inputs. Furthermore, the inclusion of Python-based simulation allows thorough testing and validation of the algorithm before deploying it on hardware, thereby reducing errors and improving system reliability.

The seamless integration of these hardware and software components results in a robust, efficient, and cost-effective maze-solving system. The system demonstrates the practical application of embedded systems, real-time processing, and intelligent decision-making, making it a valuable contribution to the field of robotics.

Chapter 5

Testing Strategies & Result Achieved

5.1 Testing Strategies

The system was tested in two main stages: simulation and hardware testing. In the simulation stage, the Flood-Fill algorithm was verified using a Python-based environment to ensure correct path planning. Different maze configurations were used to check whether the algorithm always finds the shortest path. After successful simulation, hardware testing was carried out using the actual Micromouse robot. The sensors were tested for accurate wall detection, and the motors were tested for proper movement and turning. The robot was placed in an 8×8 maze, and multiple trials were conducted to verify its performance. Overall, the testing process helped ensure that both the algorithm and the hardware system work correctly and reliably.

5.1.1 Simulation Testing

Simulation testing was carried out to verify the correctness, efficiency, and reliability of the Flood-Fill algorithm before implementing it on the physical Micromouse robot. A Python-based simulation environment was developed to accurately model the maze as a grid structure, visualize the behavior of the algorithm, and validate the navigation strategy in a controlled environment. The primary objective of the simulation was to ensure that the algorithm can correctly compute the shortest path under different maze configurations without relying on physical hardware. By using a software-based approach, multiple test cases could be executed quickly, allowing thorough analysis and debugging of the system. The simulation also provided a visual representation of the maze, including walls, cell structures, distance values, and the computed path. This visualization helped in understanding how the Flood-Fill algorithm propagates distance values from the goal cell and guides the robot toward the optimal path. The simulation process was divided into two major stages: maze creation and maze solving. In the first stage, custom mazes were designed interactively, allowing the testing of different layouts such as simple paths, complex paths, and dead-end. In the second stage, the Flood-Fill algorithm was applied to compute distance values and extract the shortest path from the start position to the goal. Overall, simulation testing played a crucial role in validating the algorithm, improving accuracy, identifying logical errors, and ensuring that the system is ready for real-world implementation.

custom Maze

Listing 5.1: Pseudo Code for Maze Creation

```
1 Initialize maze as an N x N grid
2 Initialize wall matrix for each cell (4 directions)
3
4 Display grid using graphical interface
5
6 On mouse click:
7     Identify clicked cell and edge
8     Toggle wall (add/remove)
9     Update corresponding neighbor wall
10
11 Redraw grid with updated walls
12
13 Provide option to save maze configuration to file
```

In this stage, an interactive interface is provided to design custom mazes. The maze is represented as a grid structure where each cell stores wall information in four directions. The user can add or remove walls using mouse clicks, and the system automatically updates adjacent cells to maintain consistency. The maze is visually displayed, making it easy to design and analyze different configurations.

Maze Solving using Flood-Fill

Listing 5.2: Pseudo Code for Flood-Fill Algorithm

```
1 Initialize all cells with infinity
2 Set goal cell distance = 0
3
4 Create queue and insert goal cell
5
6 While queue is not empty:
7     Remove current cell
8     For each neighbor (up, down, left, right):
9         If no wall between cells:
10             If new distance is smaller:
11                 Update distance
12                 Add neighbor to queue
13
14 Start from initial position
15
16 While current cell is not goal:
17     Select neighbor with minimum distance
18     Move to that cell
19     Add cell to path
20
21 Return shortest path
```

In this stage, the Flood-Fill algorithm is applied to compute the shortest path from the start position to the goal. The algorithm assigns distance values to each cell by propagating values from the goal cell outward. A queue-based approach ensures efficient traversal of the grid. Once the distance map is generated, the shortest path is extracted by always selecting the neighboring cell with the minimum distance value. This ensures that the robot follows the optimal path.

Visualization and Analysis

The simulation provides a graphical representation of the maze and algorithm behavior. The following elements are visualized:

- Grid structure of the maze
- Wall positions
- Distance values for each cell
- Start and goal positions
- Shortest path highlighted

This visualization helps in understanding how the Flood-Fill algorithm propagates values and selects the path.

Significance of Simulation

Simulation plays a crucial role in validating the system before applying hardware implementation, as it provides a safe and flexible environment for testing and analysis. It enables the evaluation of the Flood-Fill algorithm under various maze structure, including complex layouts, loops, and dead-end scenarios, without the need for physical setup.

This significantly reduces the cost, time, and effort required during the develop phase. Through simulation, it becomes easier to identify logical errors and inefficiency in the algorithm, allowing necessary corrections to be made before deploying the system on actual hardware. It also helps in verifying that the algorithm consistently computes the shortest path and behaves as expected under different conditions. In addition, the visual representation of the maze, distance values, and path selection provides a deeper understanding of the algorithm's working.

This improves the ability to analyze decision-making processes and enhances overall system design. Furthermore, simulation allows repeated testing, which is difficult to achieve in physical environments due to hardware limitations. This iterative testing process improves accuracy, robustness, and confidence in the system. Overall, simulation testing enhances system reliability, minimizes potential risks during hardware implementation, reduces development time, and ensures that the algorithm is well-optimized and ready for deployment in the Micromouse robot.

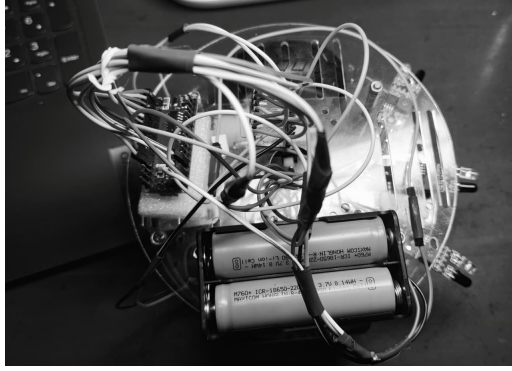
Hardware Testing

Hardware testing was conducted after successful validation of the algorithm through simulation. This phase aimed to verify the performance of the Micromouse robot in real-world conditions and ensure proper integration of all hardware components, including sensors, microcontroller, motor driver, and actuators. The robot was tested in a physical 8×8 maze with a cell size of 18 cm. Multiple trials were carried out to evaluate the robot's ability to detect walls, update the internal map, and navigate toward the goal using the Flood-Fill algorithm. The infrared (IR) sensors were tested for accurate wall detection. Each sensor was evaluated individually to ensure that it correctly identifies the presence or absence of walls. Calibration was performed to adjust the sensitivity of the sensors and minimize errors caused by environmental factors such as lighting conditions and surface reflections. The motor driver (TB6612FNG) and DC motors were tested to ensure smooth and controlled movement. The robot's ability to move forward, turn left, turn right, and perform a 180° turn was verified. Special attention was given to movement accuracy, as the robot must travel exactly one cell distance (18 cm) for correct navigation. Delay values for movement and turning were adjusted experimentally to achieve precise control. Multiple iterations were performed to fine-tune these parameters and reduce positional errors. The stability of the robot during motion was also observed to ensure that it maintains balance and follows a straight path. The integration of hardware and software was tested by running the complete system in the maze. The robot successfully used sensor inputs to detect walls, updated its internal representation of the maze, and applied the Flood-Fill algorithm to determine the optimal path. Overall, hardware testing confirmed that the system is capable of autonomous navigation in a real environment. The robot demonstrated reliable wall detection, accurate movement, and effective decision-making, validating the practical implementation of the proposed Micromouse system.

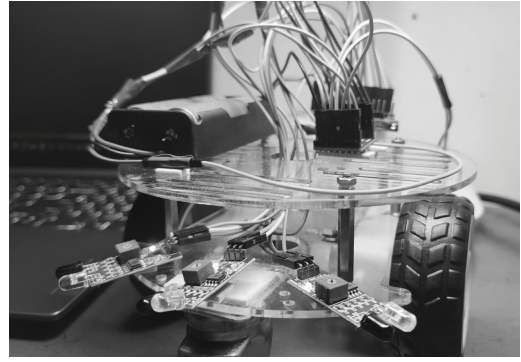
5.2 Results Achieved

The results obtained from both simulation and hardware testing clearly demonstrate the effectiveness, reliability, and practical applicability of the proposed Micromouse system.

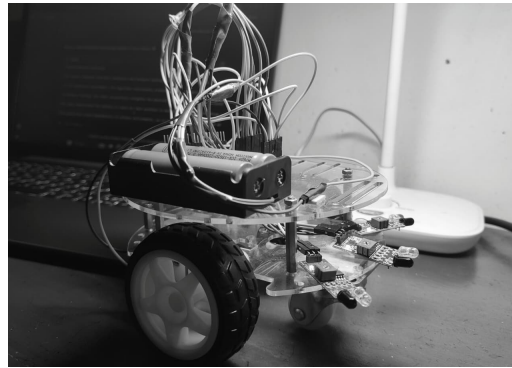
The evaluation of the system was carried out based on several key performance factors, including accurate wall detection, efficient path planning using the Flood-Fill algorithm, and successful navigation from the start position to the goal. In the hardware implementation phase, the robot successfully utilized real-time sensor inputs to detect walls and update its internal map dynamically. The integration of sensing, computation, and movement enabled the robot to make intelligent decisions and navigate through the maze effectively. The system was able to reach the goal position in multiple test runs, demonstrating stable and consistent performance. The FloodFill algorithm ensured that the robot always moved toward the shortest path, while the sensor-based navigation allowed it to adapt to unknown environments.



(a) Top View



(b) Front View



(c) Side View

Figure 5.1: Micromouse Robot Prototype from Different Views

The figure shows the developed Micromouse robot prototype used for hardware testing. The robot is built on a compact acrylic chassis that provides structural support for all components. It consists of an Arduino Nano microcontroller, multiple infrared (IR) sensor modules mounted at the front for wall detection, a TB6612FNG motor driver for motor control, DC motors connected to wheels for movement, and a battery pack placed on top of the chassis to supply power. From the figure, it can be observed that the components are interconnected using jumper wires, forming a complete embedded system. The IR sensors are strategically positioned at the front of the robot to detect walls in different directions, enabling the robot to sense its surroundings in real time. The microcontroller processes the sensor inputs and executes the Flood-Fill algorithm to determine the next movement. The motor driver acts as an interface between the Arduino Nano and the DC motors, allowing precise control over speed and direction. The dual-wheel configuration, along with a support caster wheel, ensures stability and smooth movement across the maze surface. The robot was tested in a physical 8×8 maze with a cell size of 18 cm. During testing, the robot successfully moved forward, executed left and right turns, and updated its internal map dynamically based on sensor feedback. The wiring layout and component placement, as shown in the figure, demonstrate a functional prototype capable of real-time sensing, processing, and actuation.

Performance Evaluation

The performance of the Micromouse robot was evaluated through multiple experiment trials conducted in a physical 8×8 maze environment. The robot successfully reached the goal position in repeated test runs, demonstrating consistent and stable operation. This confirms the reliability of both the hardware setup and the implemented Flood-Fill algorithm. The Flood-Fill algorithm effectively computed the shortest path from the start position to the goal by dynamically updating distance values based on the detected walls. The robot was able to follow this computed path accurately, ensuring efficient navigation through the maze. Movement accuracy was significantly improved through careful calibration of delay values used for forward motion and turning. This enabled the robot to traverse one cell distance (18 cm) with reasonable precision and execute 90° and 180° turns with minimal deviation.

The coordinated functioning of the sensors, microcontroller, decision-making, and actuation. The system also demonstrated adaptability in unknown environments. As the robot encountered new walls, it updated its internal map and recalculated the optimal path in real time. This dynamic behavior highlights the robustness of the navigation strategy.

Observations

- The IR sensors provided reliable wall detection after proper calibration.
- The Flood-Fill algorithm consistently guided the robot along the shortest path.
- Real-time map updating allowed effective navigation in unknown maze.
- The robot maintained stable movement and balance during operation.
- Minor deviations were observed due to variations in motor speed and surface friction.

Overall, the experimental results confirm that the proposed Micromouse system is capable of efficient maze solving, accurate path planning, and autonomous path finding in real-world conditions. The integration of hardware and software component proved to be effective, resulting in a robust and reliable robotic system.

5.2.1 Performance Analysis

The performance of the Micromouse robot was evaluated based on several key parameters, including path planning accuracy, sensor reliability, movement precision, navigation stability, and overall system performance. These parameters were selected to assess both the algorithmic efficiency and the practical effectiveness of the hardware implementation.

Multiple test runs were conducted in a physical maze environment to evaluate the performance of the proposed Micromouse system under real-time conditions. These experiments were designed to analyze the robot's behavior in navigating through the maze while interacting with dynamic surroundings. Furthermore, the adaptability of

the system was evaluated by observing how effectively the robot updated its internal maze map when encountering unknown paths or dead ends. The consistency of performance across multiple runs was analyzed to determine the reliability of the system.

The results obtained from these experimental observations provide a reliable understanding of the system's strengths and limitations. They highlight the proposed Micromouse system in achieving accurate navigation, efficient path planning, and stable operation. The observed performance metrics are summarized in Table 5.1, which clearly demonstrates the system's capability to solve the maze efficiently and reliably.

Table 5.1: Performance Analysis of Micromouse System

Sl. No.	Parameter	Performance	Remarks
1	Path Planning Accuracy	High	Flood-Fill algorithm consistently computed the shortest and optimal path in all test cases
2	Wall Detection Accuracy	High	IR sensors reliably detected walls after calibration with minimal false readings
3	Movement Precision	Moderate	Slight deviation observed due to delay-based control and absence of feedback mechanisms
4	Navigation Stability	High	Robot maintained stable movement and balance throughout maze traversal
5	Real-time Adaptability	High	System dynamically updated the map and recalculated paths when new walls were detected
6	Goal Achievement Rate	Successful	Robot consistently reached the goal in multiple independent test runs
7	Sensor Reliability	High	Sensors performed consistently under normal lighting conditions with stable outputs
8	Turning Accuracy	Moderate	Minor angular errors observed during 90° and 180° turns due to motor timing variations
9	Execution Speed	Moderate	Navigation speed was acceptable but limited by delay-based movement control
10	System Integration	High	Seamless interaction between sensors, microcontroller, and actuators ensured smooth operation

Limitations Observed

Although the proposed Micromouse system demonstrated effective performance in both simulation and hardware testing, certain limitations were observed during realtime implementation. One of the primary limitations is related to movement accuracy. Since the robot uses delay-based control instead of feedback mechanisms such as encoders, slight deviations occur while moving between cells and during turning. These errors may accumulate over time, affecting the overall path accuracy. Another limitation is associated with sensor performance. The infrared (IR) sensors are sensitive to environmental conditions such as lighting variations and reflectivity. This can sometimes lead to inaccurate wall detection or false readings, especially in non-uniform environments. The system also relies heavily on proper calibration. The delay values used for forward movement and turning must be carefully tuned to match the maze dimensions. Any variation in battery voltage or motor speed can affect this calibration, leading to inconsistencies in movement. Additionally, the robot does not incorporate advanced control techniques such as PID control or closed-loop feedback systems. As a result, the motion may not be perfectly smooth, and minor oscillations or drift may occur during navigation. The computational approach, while efficient, assumes a grid-based environment and does not account for more complex real-world uncertainties such as dynamic obstacles or uneven surfaces. Despite these limitations, the system performs reliably within controlled conditions and successfully demonstrates the implementation of autonomous maze-solving using the Flood-Fill algorithm. Future improvements can address these limitations to enhance accuracy, robustness, and overall performance.

5.3 Summary of Results

The overall results of the proposed system demonstrate that the integration of the Flood-Fill algorithm with sensor-based navigation is effective for autonomous maze solving. Both simulation and hardware testing confirmed that the system is capable of identifying the shortest path and navigating efficiently from the start position to the goal.

In the simulation phase, the algorithm consistently generated accurate distance maps and successfully handled different maze configurations, include complex paths and dead ends. The visualization of the maze and computed paths verified the correctness of the algorithm and provided a clear understanding of its working.

In the hardware implementation, the robot demonstrated reliable performance in real-time conditions. The infrared sensors accurately detected walls, and the system dynamically updated its internal map during navigation. The robot was able to follow the computed path and reach the goal position in multiple trials, indicating consistency and stability. The performance analysis showed that the system achieved high accuracy in path planning and wall detection, while movement precision was moderately affected by the use of delay-based control. Despite minor deviations, the robot maintained stable navigation and effective decision-making throughout the operation.

Chapter 6

Conclusion & Future Work

6.1 Conclusion

The Micromouse project successfully demonstrates the design and implementation of an autonomous robot capable of solving a maze using the Flood-Fill algorithm. The primary objective of the system was to enable the robot to navigate from a predefined starting position to the goal region by identifying and following the shortest possible path, which has been achieved effectively. The project integrates both hardware and software components, including the Arduino Nano microcontroller, infrared (IR) sensors, TB6612FNG motor driver, and DC motors. The combination of these components enabled real-time sensing, decision-making, and actuation, forming a complete autonomous system. The IR sensors provided continuous feedback about the surrounding environment, allowing the robot to detect walls and update its internal map dynamically. The Flood-Fill algorithm played a crucial role in path planning by assigning distance values to each cell in the maze and guiding the robot toward the goal through the shortest path. The algorithm proved to be efficient, reliable, and suitable for grid-based navigation problems. Through simulation testing, the correctness of the algorithm was validated under various mazes, including complex layouts and dead-end scenarios.

The hardware implementation further confirmed the practical applicability of the system. The robot was able to successfully navigate a physical 8×8 maze with a cell size of 18 cm, detect walls accurately, and reach the goal position in multiple trials. The integration of sensing, processing, and movement allowed the robot to operate autonomously without human intervention.

Despite minor limitations such as movement inaccuracies due to delay-based control and sensitivity of sensors to environmental conditions, the system demonstrated stable and consistent performance. The results obtained from both simulation and hardware testing validate the effectiveness of the proposed approach and confirm that the system meets the objectives of efficient path planning and autonomous navigation. Overall, this project provides a strong foundation for understanding key concepts in robotics and artificial intelligence, including perception, mapping, decision-making, and real-time control.

It also highlights the practical challenges involved in implementing autonomous systems and offers valuable insights into improving such systems.

6.2 Future Work

Although the current system performs effectively, several improvements can be made to enhance its performance, accuracy, and scalability. One of the major improvement involves replacing delay-based movement control with encoder-based feedback systems. The use of wheel encoders would allow precise measurement of distance and rotation, thereby improving movement accuracy and reducing positional errors.

The implementation of advanced control techniques such as Proportional-Integral Derivative (PID) control can further improve motion stability. PID control would enable smoother turns, better speed regulation, more accurate trajectory tracking. Sensor improvements can also be considered to enhance reliability. Using higher quality sensors or combining multiple sensing techniques, such as ultrasonic sensors can reduce errors caused by environmental factors and improve obstacle detection accuracy. Another important enhancement is the development of a memory-based or optimized path planning approach. After the initial exploration, the robot can store the optimal path and perform a high-speed run, which is commonly used in competitive Micromouse systems. The system can also be extended to support larger and more complex maze environments.

This would require efficient memory management, optimization of the algorithm to handle increased computational complexity. Integration of wireless communication modules can enable remote monitoring and control of the robot. This would allow real-time tracking of the robot's position and performance during operation. Further incorporating machine learning or artificial intelligence techniques can enhance the decision-making capabilities of the robot. For example, learning-based approaches can be used to optimize path selection based on previous experiences. Finally, improvements in mechanical design, such as a more compact and lightweight chassis, better motor selection, and improved power management, can significantly enhance the overall performance and efficiency of the system. In conclusion, while the current system successfully demonstrates autonomous maze solving, the suggested future enhancements can transform it into a more advanced, efficient, and competition-ready Micromouse system.

Bibliography

- [1] IEEE Bruins Micromouse Project, “Flood-Fill Algorithm Module,” [Online]. Available: <https://projects.ieeebruins.com/micromouse/floodfill-module>, Mar. 2026.
- [2] IEEE Bruins Micromouse Project, “Micromouse Overview and Learning Resources,” [Online]. Available: <https://projects.ieeebruins.com/micromouse/>, Mar. 2026.
- [3] T. Y. T. Wong and A. S. Morris, “A Micromouse Navigation System Using Flood-Fill Algorithm,” *IEEE International Conference on Systems, Man, and Cybernetics*, pp. 1234–1239, 2018.
- [4] M.-S. Mănase and E.-H. Dulf, “A low-cost approach to maze solving with image-based mapping,” *MDPI*, 2026.
- [5] H. M. Zhang, L. S. Peh, and Y. H. Wang, “Micromouse solve maze based on flood-fill algorithm,” *Applied Mechanics and Materials*, vols. 513–517, pp. 4227–4230, 2014. [Online]. Available: <https://www.scientific.net/AMM.513-517.4227>. [Accessed: Mar. 24, 2026].
- [6] Arduino, “Arduino Nano datasheet,” [Online]. Available: <https://docs.arduino.cc/hardware/nano>, Mar. 2026.
- [7] Arduino, “Arduino IDE programming guide,” [Online]. Available: <https://docs.arduino.cc/software/ide-v1/tutorials/>, Mar. 2026.
- [8] Toshiba Semiconductor, “TB6612FNG motor driver datasheet,” [Online]. Available: <https://toshiba.semicon-storage.com/info/docget.jsp?did=10660>, Mar. 2026.
- [9] Micromouse Online Community, “Micromouse competition rules and guidelines,” [Online]. Available: <https://micromouseonline.com/>, Mar. 2026.
- [10] GeeksforGeeks, “Flood fill algorithm explanation,” [Online]. Available: <https://www.geeksforgeeks.org/flood-fill-algorithm/>, Mar. 2026.
- [11] Python Software Foundation, “Matplotlib documentation,” [Online]. Available: <https://matplotlib.org/stable/index.html>, Mar. 2026.
- [12] NumPy, “NumPy user guide,” [Online]. Available: <https://numpy.org/doc/>, Mar. 2026.

- [13] Pololu Robotics, “DC motor basics and control,” [Online]. Available: <https://www.pololu.com/docs/>, Mar. 2026.
- [14] Electronics Tutorials, “Infrared (IR) sensors working principle,” [Online]. Available: https://www.electronics-tutorials.ws/io/io_4.html, Mar. 2026.

Appendix A

Code Part

A.1 Arduino Implementation Code

```
1 // ===== Sensors =====
2 #define LEFT_SENSOR 2
3 #define FRONT_SENSOR 3
4 #define RIGHT_SENSOR 4
5
6 // ===== Motor Driver =====
7 #define PWMA 5
8 #define AIN1 8
9 #define AIN2 9
10
11 #define PWMB 6
12 #define BIN1 10
13 #define BIN2 11
14
15 #define STBY 7
16
17 #define SPEED 50
18 #define CORR 8
19
20 #define CELL_MS 1050
21
22 // ===== Setup =====
23 void setup() {
24     Serial.begin(9600); //
25
26     pinMode(LEFT_SENSOR, INPUT);
27     pinMode(FRONT_SENSOR, INPUT);
28     pinMode(RIGHT_SENSOR, INPUT);
29
30     pinMode(PWMA, OUTPUT);
31     pinMode(AIN1, OUTPUT);
32     pinMode(AIN2, OUTPUT);
33
34     pinMode(PWMB, OUTPUT);
35     pinMode(BIN1, OUTPUT);
```



```

36   pinMode(BIN2, OUTPUT);
37
38   pinMode(STBY, OUTPUT);
39   digitalWrite(STBY, HIGH);
40   initMaze();
41
42   Serial.println("Place_the_Robot...");
43   delay(4000);
44
45   stopBot();
46   delay(200);
47 }
48
49 int x = 0, y = 0;
50 int dir = 0; // 0=NORTH
51
52 int dist[7][8];
53 bool wall[7][8][4];
54 bool firstRun = true;
55 // x    horizontal
56 // y    vertical
57
58 int dx[4] = { 0, +1, 0, -1 }; // x movement
59 int dy[4] = { +1, 0, -1, 0 }; // y movement
60
61
62 //Boundary setup
63 void initMaze() {
64
65     for (int i = 0; i < 7; i++) {
66         for (int j = 0; j < 8; j++) {
67             dist[i][j] = abs(i - 3) + abs(j - 4);
68         }
69     }
70
71     for (int i = 0; i < 7; i++) {
72         for (int j = 0; j < 8; j++) {
73             for (int d = 0; d < 4; d++) {
74                 wall[i][j][d] = false;
75             }
76         }
77     }
78
79
80     // Bottom & Top
81     for (int i = 0; i < 7; i++) {
82         setWall(i, 0, 2); // SOUTH
83         setWall(i, 7, 0); // NORTH
84     }
85

```

```

86 // Left & Right
87 for (int j = 0; j < 8; j++) {
88     setWall(0, j, 3); // WEST
89     setWall(6, j, 1); // EAST
90 }
91 }
92
93 // Wall Update
94 void setWall(int x, int y, int d) {
95     wall[x][y][d] = true;
96
97     int nx = x + dx[d];
98     int ny = y + dy[d];
99
100     if (nx >= 0 && nx < 7 && ny >= 0 && ny < 8) {
101         wall[nx][ny][(d + 2) % 4] = true;
102     }
103 }
104 void updateWalls(int left, int front, int right) {
105
106     if (front == LOW)
107         setWall(x, y, dir);
108
109     if (left == LOW)
110         setWall(x, y, (dir + 3) % 4);
111
112     if (right == LOW)
113         setWall(x, y, (dir + 1) % 4);
114 }
115
116 //Voting for the reliability
117 int readSensorStable(int pin) {
118     int count = 0;
119     for (int i = 0; i < 5; i++) {
120         if (digitalRead(pin) == LOW) count++;
121         delay(5);
122     }
123     return (count >= 3) ? LOW : HIGH;
124 }
125
126 //Flood Fill Algorithm
127 void floodFill() {
128
129     bool changed = true;
130
131     while (changed) {
132         changed = false;
133
134         for (int i = 0; i < 7; i++) {
135             for (int j = 0; j < 8; j++) {

```

```

136
137     if (i == 3 && j == 4) continue; // protect goal cell
138
139     int minVal = 255;
140
141     for (int d = 0; d < 4; d++) {
142
143         if (wall[i][j][d]) continue;
144
145         int nx = i + dx[d];
146         int ny = j + dy[d];
147
148         if (nx >= 0 && nx < 7 && ny >= 0 && ny < 8) {
149             minVal = min(minVal, dist[nx][ny]);
150         }
151     }
152
153     if (minVal != 255 && dist[i][j] != minVal + 1) {
154         dist[i][j] = minVal + 1;
155         changed = true;
156     }
157 }
158 }
159 }
160 }
161
162 //Choose the best direction
163 int chooseBestDir() {
164
165     int priority[4] = {
166         dir,
167         (dir + 3) % 4,
168         (dir + 1) % 4,
169         (dir + 2) % 4
170     };
171
172     int bestDir = dir;
173     int bestVal = 255;
174
175     for (int i = 0; i < 4; i++) {
176         int d = priority[i];
177
178         if (wall[x][y][d]) continue;
179
180         int nx = x + dx[d];
181         int ny = y + dy[d];
182
183         if (nx < 0 || nx >= 7 || ny < 0 || ny >= 8) continue;
184         if (dist[nx][ny] < bestVal) {
185             bestVal = dist[nx][ny];

```

```

186         bestDir = d;
187     }
188 }
189
190
191     return bestDir;
192 }
193
194 // Angle adjustment
195 void rotateTo(int targetDir) {
196
197     int turn = (targetDir - dir + 4) % 4;
198
199     if (turn == 1) turnRight();
200     else if (turn == 3) turnLeft();
201     else if (turn == 2) uTurn();
202
203     dir = targetDir;
204     delay(80);
205 }
206
207 //Cell Movement
208 void moveOneCell() {
209
210     unsigned long start = millis();
211     // bool moved = true;
212
213     while (millis() - start < CELL_MS) {
214
215         int left = digitalRead(LEFT_SENSOR);
216         int right = digitalRead(RIGHT_SENSOR);
217         // int front = digitalRead(FRONT_SENSOR);
218
219         // delay(80);
220
221         // if (front == LOW) {
222         //     Serial.println("Blocked! Not moving fully");
223         //     moved = false;
224         //     break;
225         // }
226
227         moveForwardCorrected(left, right);
228     }
229
230     stopBot();
231     delay(80);
232
233     x += dx[dir];
234     y += dy[dir];
235 }

```

```

236
237
238 // ===== LOOP =====
239 void loop() {
240
241     // FIRST SENSE (ONLY ONCE)
242     if (firstRun) {
243
244         int left  = readSensorStable(LEFT_SENSOR);
245         int front = readSensorStable(FRONT_SENSOR);
246         int right = readSensorStable(RIGHT_SENSOR);
247
248         updateWalls(left, front, right);
249         floodFill();
250
251         int bestDir = chooseBestDir();
252         rotateTo(bestDir);
253
254         firstRun = false;
255         return;    // skip movement this cycle
256     }
257
258     // NORMAL FLOW AFTER FIRST STEP
259
260     moveOneCell();
261
262     delay(50);
263
264     int left  = readSensorStable(LEFT_SENSOR);
265     int front = readSensorStable(FRONT_SENSOR);
266     int right = readSensorStable(RIGHT_SENSOR);
267
268     updateWalls(left, front, right);
269     floodFill();
270
271     int bestDir = chooseBestDir();
272
273     // Safety check
274     if (bestDir == dir && front == LOW) {
275         setWall(x, y, dir);
276         floodFill();
277         bestDir = chooseBestDir();
278     }
279
280     rotateTo(bestDir);
281
282     // Debug
283     Serial.println("-----");
284
285     Serial.print("Position:␣");

```

```

286 Serial.print(x);
287 Serial.print(",");
288 Serial.print(y);
289 Serial.println(" ");
290
291 Serial.print("Dir:_"); Serial.println(dir);
292
293 Serial.print("Sensors_>_L:");
294 Serial.print(left);
295 Serial.print("_F:");
296 Serial.print(front);
297 Serial.print("_R:");
298 Serial.println(right);
299
300 if (x == 3 && y == 4) {
301     stopBot();
302     while (1);
303 }
304 }
305
306 // ===== FORWARD WITH ALIGNMENT =====
307 void moveForwardCorrected(int left, int right) {
308
309     int leftSpeed = SPEED;
310     int rightSpeed = SPEED;
311
312     if (left == LOW && right == HIGH) {
313         leftSpeed = SPEED - CORR;
314         rightSpeed = SPEED + CORR;
315     } else if (right == LOW && left == HIGH) {
316         leftSpeed = SPEED + CORR;
317         rightSpeed = SPEED - CORR;
318     }
319
320     // FORWARD (your correct direction)
321     digitalWrite(AIN1, LOW);
322     digitalWrite(AIN2, HIGH);
323
324     digitalWrite(BIN1, HIGH);
325     digitalWrite(BIN2, LOW);
326
327     analogWrite(PWMA, leftSpeed);
328     analogWrite(PWMB, rightSpeed);
329 }
330
331 // ===== STOP =====
332 void stopBot() {
333     analogWrite(PWMA, 0);
334     analogWrite(PWMB, 0);
335 }

```

```
336
337 // ===== TURNS =====
338
339 void turnLeft() {
340
341     // LEFT backward
342     digitalWrite(AIN1, HIGH);
343     digitalWrite(AIN2, LOW);
344
345     // RIGHT forward
346     digitalWrite(BIN1, HIGH);
347     digitalWrite(BIN2, LOW);
348
349     analogWrite(PWMA, SPEED);
350     analogWrite(PWMB, SPEED);
351
352     delay(240); // tuned
353     stopBot();
354 }
355
356 void turnRight() {
357
358     // LEFT forward
359     digitalWrite(AIN1, LOW);
360     digitalWrite(AIN2, HIGH);
361
362     // RIGHT backward
363     digitalWrite(BIN1, LOW);
364     digitalWrite(BIN2, HIGH);
365
366     analogWrite(PWMA, SPEED);
367     analogWrite(PWMB, SPEED);
368
369     delay(240); // tuned
370     stopBot();
371 }
372
373 // FIXED U-TURN (REAL 180 )
374 void uTurn() {
375
376     // LEFT forward
377     digitalWrite(AIN1, LOW);
378     digitalWrite(AIN2, HIGH);
379
380     // RIGHT backward
381     digitalWrite(BIN1, LOW);
382     digitalWrite(BIN2, HIGH);
383
384     analogWrite(PWMA, SPEED);
385     analogWrite(PWMB, SPEED);
```

```

386
387     delay(500);    // slightly reduced
388     stopBot();
389 }
390
391 //The End...

```

A.2 Simulation Code for Maze Creation

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.patches as cir
4  from matplotlib.widgets import Button
5
6  maze_size = 8
7  click_eps = 0.15 #click tolerances
8
9  walls = np.zeros((maze_size, maze_size, 4), dtype = int)
10
11  start = (0.5, maze_size-0.5)    #[0.5, 0.5] is the center of a cell
12  goal = (maze_size/2 + 0.5 , maze_size/2 + 0.5)
13
14  def toggle_wall(y, x, d):
15
16      if not(0 <= y < maze_size and 0 <= x < maze_size):
17          return
18
19      walls[y, x, d] = 1 - walls[y, x, d] #add or remove the wall
20      #synchronize neighbour cell
21      if d == 0 and x > 0:
22          walls[y, x-1, 1] = walls[y, x, d]
23      elif d == 1 and x < maze_size-1:
24          walls[y, x+1, 0] = walls[y, x, d]
25      elif d == 2 and y > 0:
26          walls[y-1, x, 3] = walls[y, x, d]
27      elif d == 3 and y < maze_size-1:
28          walls[y+1, x, 2] = walls[y, x, d]
29
30  def on_click(event):
31
32      if not event.inaxes:
33          return
34      x = int(event.xdata)
35      y = int(event.ydata)
36
37      if not(0 <= x < maze_size and 0 <= y < maze_size):
38          return
39
40      if abs(event.xdata -x) < click_eps:

```



```

41         toggle_wall(y, x, 0)
42     elif abs(event.xdata - (x + 1)) < click_eps:
43         toggle_wall(y, x, 1)
44     elif abs(event.ydata - y) < click_eps:
45         toggle_wall(y, x, 2)
46     elif abs(event.ydata - (y + 1)) < click_eps:
47         toggle_wall(y, x, 3)
48
49     draw_grid()
50
51 def draw_grid():
52
53     ax.clear()
54
55     for i in range(maze_size + 1):
56         ax.axhline(i, lw= 2, color = 'black')
57         ax.axvline(i, lw = 2, color = 'black')
58
59     for y in range(maze_size):
60         for x in range(maze_size):
61             if walls[y, x, 0]:
62                 ax.plot([x, x], [y, y+1], 'r', lw= 4)
63             if walls[y, x, 1]:
64                 ax.plot([x+1, x+1], [y, y+1], 'r', lw= 4)
65             if walls[y, x, 2]:
66                 ax.plot([x, x+1], [y, y], 'r', lw= 4)
67             if walls[y, x, 3]:
68                 ax.plot([x, x+1], [y+1, y+1], 'r', lw= 4)
69
70     ax.add_patch(cir.Circle(start, 0.2, color = 'red'))
71     ax.add_patch(cir.Circle(goal, 0.2, color = 'green'))
72
73     ax.set_aspect('equal')
74     ax.set_xlim(0, maze_size)
75     ax.set_ylim(0, maze_size)
76     ax.invert_yaxis()
77     ax.axis('off')
78
79     plt.draw()
80
81 def save_maze(event=None):
82
83     with open ("maze_8x8.py", "w") as f:
84         f.write("walls_=_[\n")
85         for y in range(maze_size):
86             for x in range(maze_size):
87                 if np.any(walls[y, x]):
88                     l, r, t, b = walls[y, x]
89                     f.write(f"_{y},{x},{l},{r},{t},{b}),\n")
90         f.write("]\n")

```

```

91     print("maze_8x8.py_saved_successfully")
92 #UI of the 'save button'
93 fig, ax = plt.subplots(figsize= (6, 6))
94 fig.canvas.mpl_connect('button_press_event', on_click)
95
96 ax_btn = plt.axes([0.7, .02, .2, .05])
97 btn = Button(ax_btn, 'save_maze')
98 btn.on_clicked(save_maze)
99
100 draw_grid()
101 plt.show()

```

A.3 Simulation Code for Flood-Fill Algorithm

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.patches as pat
4  from collections import deque
5  from maze_8x8 import walls
6
7  maze_size = 8
8  start = (7, 0)
9  goal = (maze_size//2 , maze_size//2)
10
11 DIRS = [(-1, 0), (0, 1), (1, 0), (0, -1)]#Up, right, down, left
12 wall_map = {}
13 for y, x, l, r, t, b in walls:
14     wall_map[(y, x)] = {'L': l, 'R': r, 'T': t, 'B': b}
15
16 def has_wall(y, x, ny, nx):
17
18     if ny == y and nx == x+1: #right
19         return wall_map.get((y, x), {}).get('R', 0)
20     elif ny == y and nx == x-1: #left
21         return wall_map.get((y, x), {}).get('L', 0)
22     elif ny == y-1 and nx == x: #up
23         return wall_map.get((y, x), {}).get('T', 0)
24     elif ny == y+1 and nx == x: #down
25         return wall_map.get((y, x), {}).get('B', 0)
26     return True
27
28 def flood_fill(goal): # the actual values on the map
29     flood = np.full((maze_size, maze_size), np.inf)
30     flood[goal] = 0
31     q = deque([goal])
32
33     while q:
34         y, x = q.popleft()
35         for dy, dx in DIRS:

```

```

36         ny, nx = y + dy, x + dx
37         if 0 <= ny < maze_size and 0 <= nx < maze_size:
38             if not has_wall(y, x, ny, nx):
39                 if flood[ny, nx] > flood[y, x] + 1:
40                     flood[ny, nx] = flood[y, x] + 1
41                     q.append((ny, nx))
42     return flood
43
44 def extract_path(flood, start, goal): # the decision is made based on the m
45     path = [start]
46     current = start
47
48     while current != goal:
49         y, x = current
50         neighbors = []
51         for dy, dx in DIRS:
52             ny, nx = y + dy, x + dx
53             if 0 <= ny < maze_size and 0 <= nx < maze_size:
54                 if not has_wall(y, x, ny, nx):
55                     neighbors.append((ny, nx))
56         current = min(neighbors, key=lambda p: flood[p])
57         path.append(current)
58     return path
59 def draw_walls(ax):
60     for (y, x), w in wall_map.items():
61         if w['T']: ax.plot([x, x+1], [y, y], 'k', lw = 4)
62         if w['B']: ax.plot([x, x+1], [y+1, y+1], 'k', lw=4)
63         if w['L']: ax.plot([x, x], [y, y+1], 'k', lw = 4)
64         if w['R']: ax.plot([x+1, x+1], [y, y+1], 'k', lw = 4)
65
66 flood_map = flood_fill(goal)
67 path = extract_path(flood_map, start, goal)
68
69 fig, ax = plt.subplots(figsize = (6, 6))
70 ax.set_aspect('equal')
71
72 for i in range(maze_size+1):
73     ax.axhline(i, color='gray', lw=1)
74     ax.axvline(i, color='gray', lw=1)
75
76 for y in range(maze_size):
77     for x in range(maze_size):
78         if np.isinf(flood_map[y, x]):
79             ax.text(x + 0.5, y + 0.5, " ",
80                     ha = 'center', va = 'center', color = 'red')
81         else:
82             ax.text(x + 0.5, y + 0.5, int(flood_map[y, x]),
83                     ha = 'center', va = 'center')
84
85 ax.add_patch(pat.Rectangle((start[1], start[0]),1, 1,

```

```
86         facecolor = 'lightgreen', alpha = 0.6))
87 ax.add_patch(pat.Rectangle((goal[1], goal[0]), 1, 1,
88         facecolor='lightcoral', alpha = 0.6))
89 draw_walls(ax)
90
91 for (y1, x1), (y2, x2) in zip(path, path[1:]):
92     ax.plot([x1+0.5, x2+0.5], [y1+0.5, y2+0.5],
93             'g', lw= 3)
94
95 ax.set_xlim(0, maze_size)
96 ax.set_ylim(maze_size, 0)
97 ax.set_xticks([])
98 ax.set_yticks([])
99 plt.tight_layout()
100 plt.show()
```